GEORG-AUGUST-UNIVERSITÄT GÖTTINGEN IN PUBLICA COMMODA SEIT 1737

COMPUTATIONAL CELL ANALYTICS

Deep Learning for Computer Vision

Lecture 6: Object Detection & Instance Segmentation

Constantin Pape

Institute of Computer Science, Georg August Universität Göttingen

@cppape.bsky.social

https://user.informatik.uni-goettingen.de/~pape41/

Agenda for today

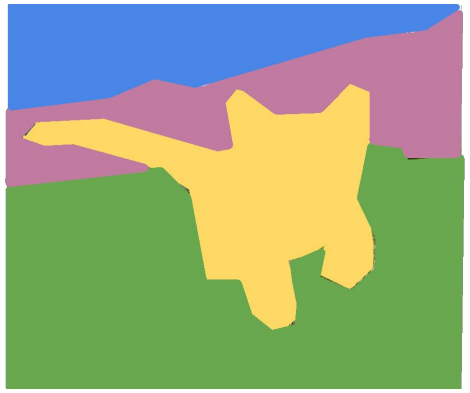
- Object detection
- Instance segmentation
- Image-to-image tasks
  - Denoising
  - Super-resolution

Segmentation tasks

Image Classification

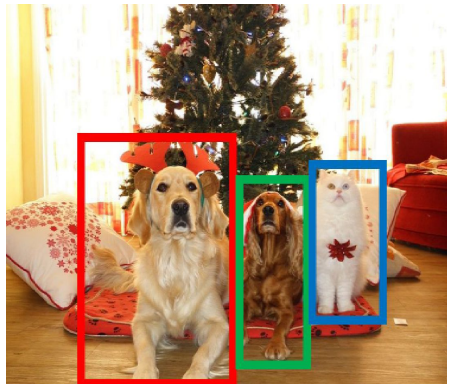
Cat

Semantic Segmentation



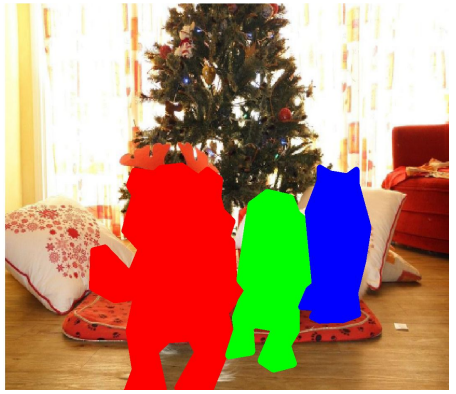
Sky, Trees, Grass, Cat

Object Detection



Dog, Dog, Cat

Instance Segmentation



Dog, Dog, Cat

No spatial extent

No objects, just pixels

Multiple objects

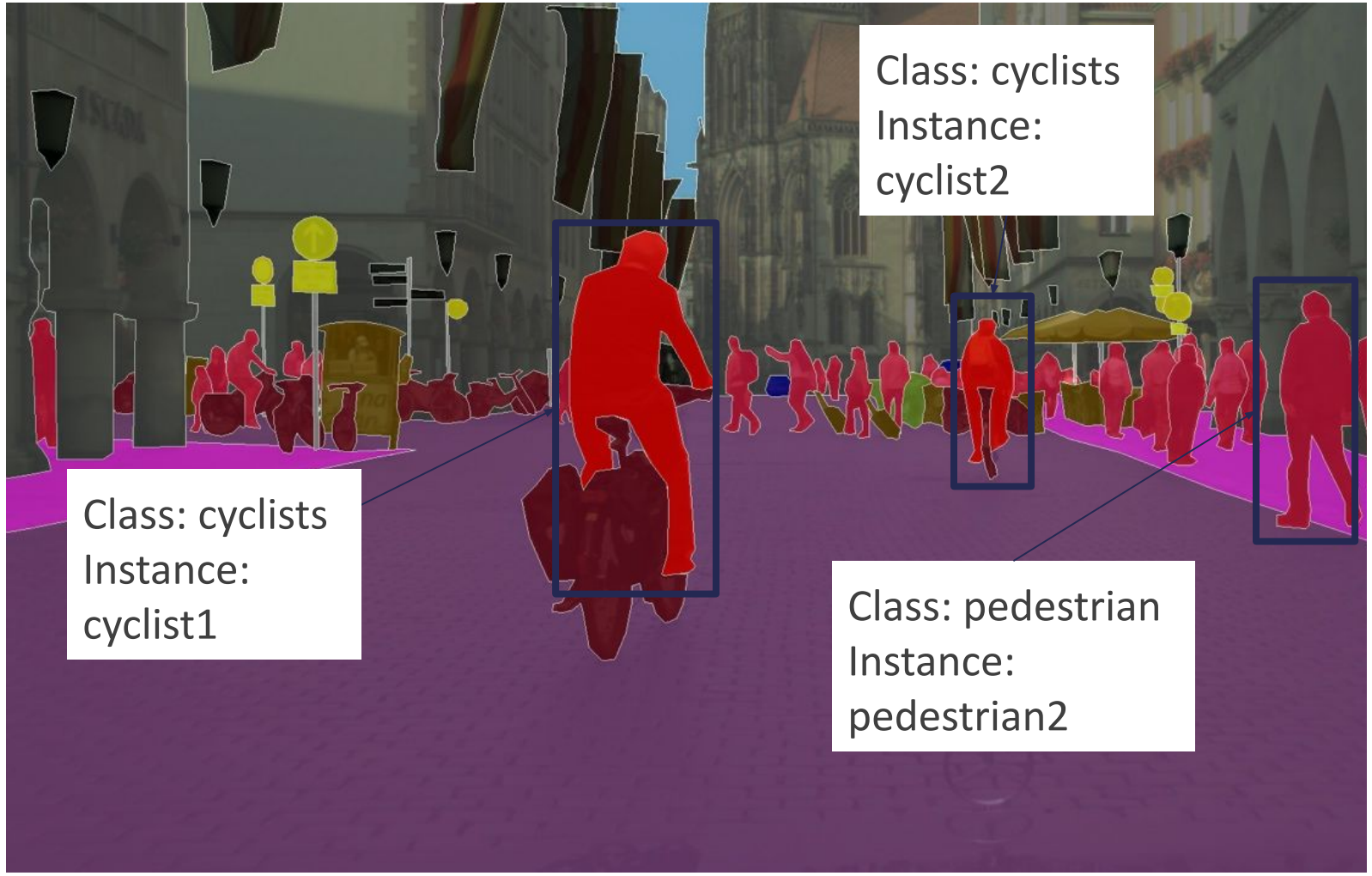
Source: Fei Fei Li, Justin Johnson, Serena Yeung

Datasets and Challenges

Many datasets combine semantic segmentation, object detection and instance segmentation.

Popular datasets:

- Pascal VOC
- MS COCO
- Cityscapes



Object Detection

Object detection

Goal:

For each object in the image, predict

- Bounding box (4 coordinates)
  - Typically: x, y, height, width
- Object class

Main problem:

How many objects are there?

Object detection

Representation:

- Position of object bounding box
- One class label for each object

Object 1:
    Bounding Box: x: 20, y: 0, h: 200, w: 100
    Class: Dog
Object 2:
    Bounding Box: x: 120, y: 20, h: 100, w: 40
    Class: Dog
Object 3:
    Bounding Box: x: 210, y: 30, h: 100, w: 45
    Class: Cat

Object detection

Object detection methods: two different approaches

- Two stage: separate networks for region proposals (possible object locations) and classification
  - R-CNN Family
  - **Faster R-CNN**

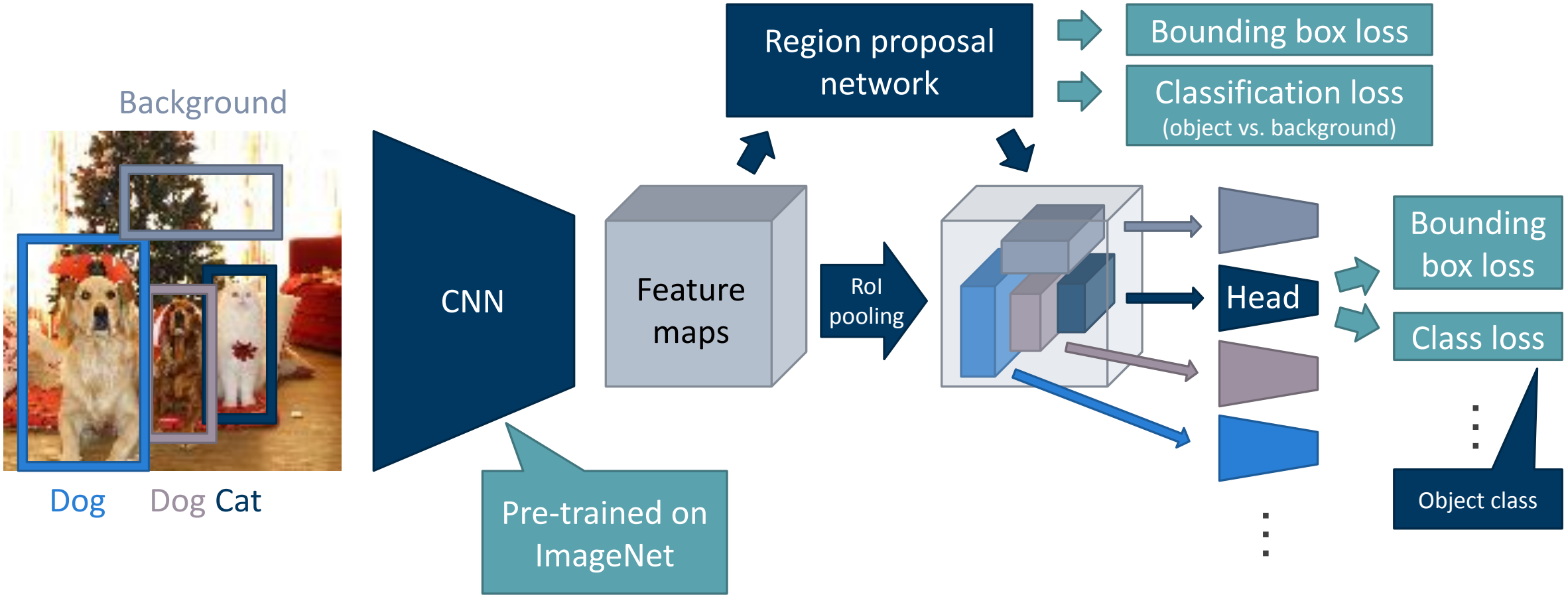
Object detection

Object detection methods: two different approaches

- Two stage: separate networks for region proposals (possible object locations) and classification
  - R-CNN Family
  - **Faster R-CNN**

- Single stage: fixed region proposals, simple network for classification and position regression
  - Yolo
  - **RetinaNet**
  - SSD

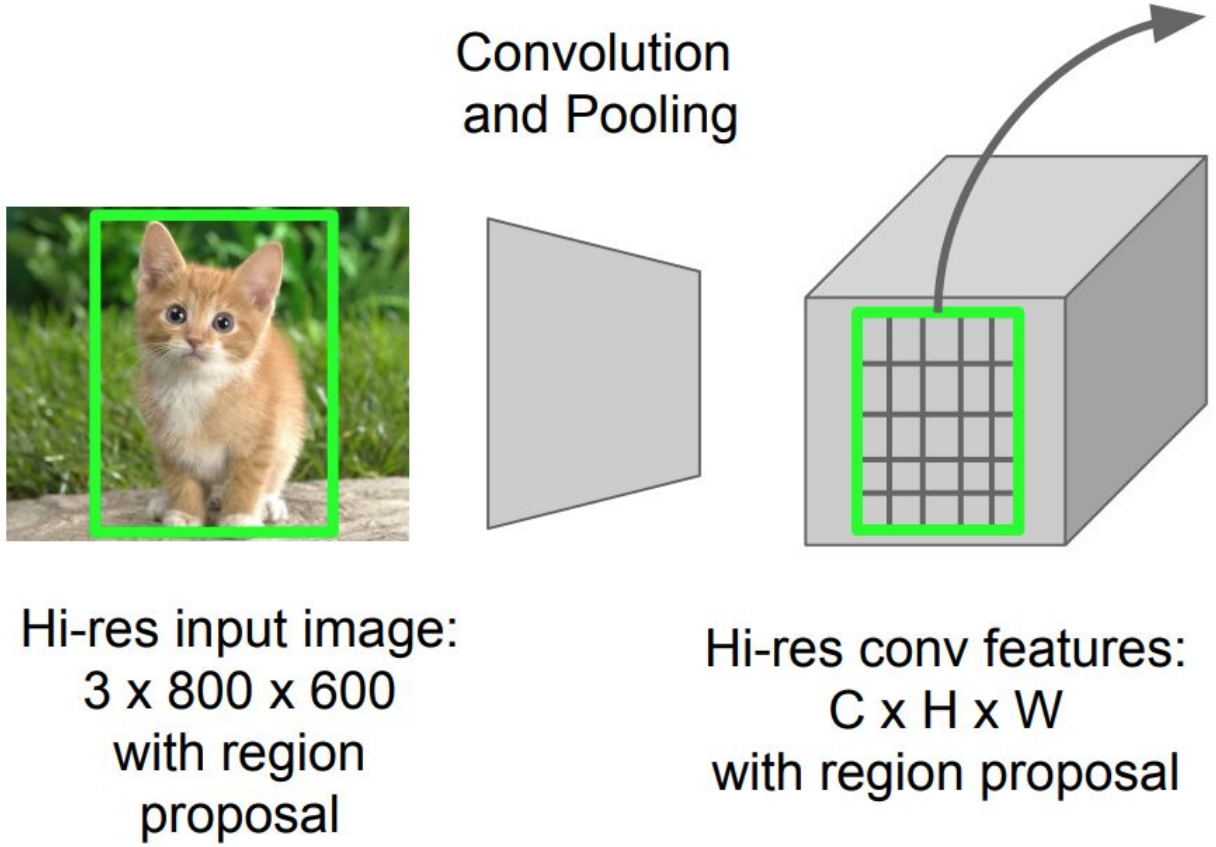
Object detection: Faster R-CNN



Ren et al., *NeurIPS* 2015

Problem: region proposals have different sizes, but Head need fixed size.

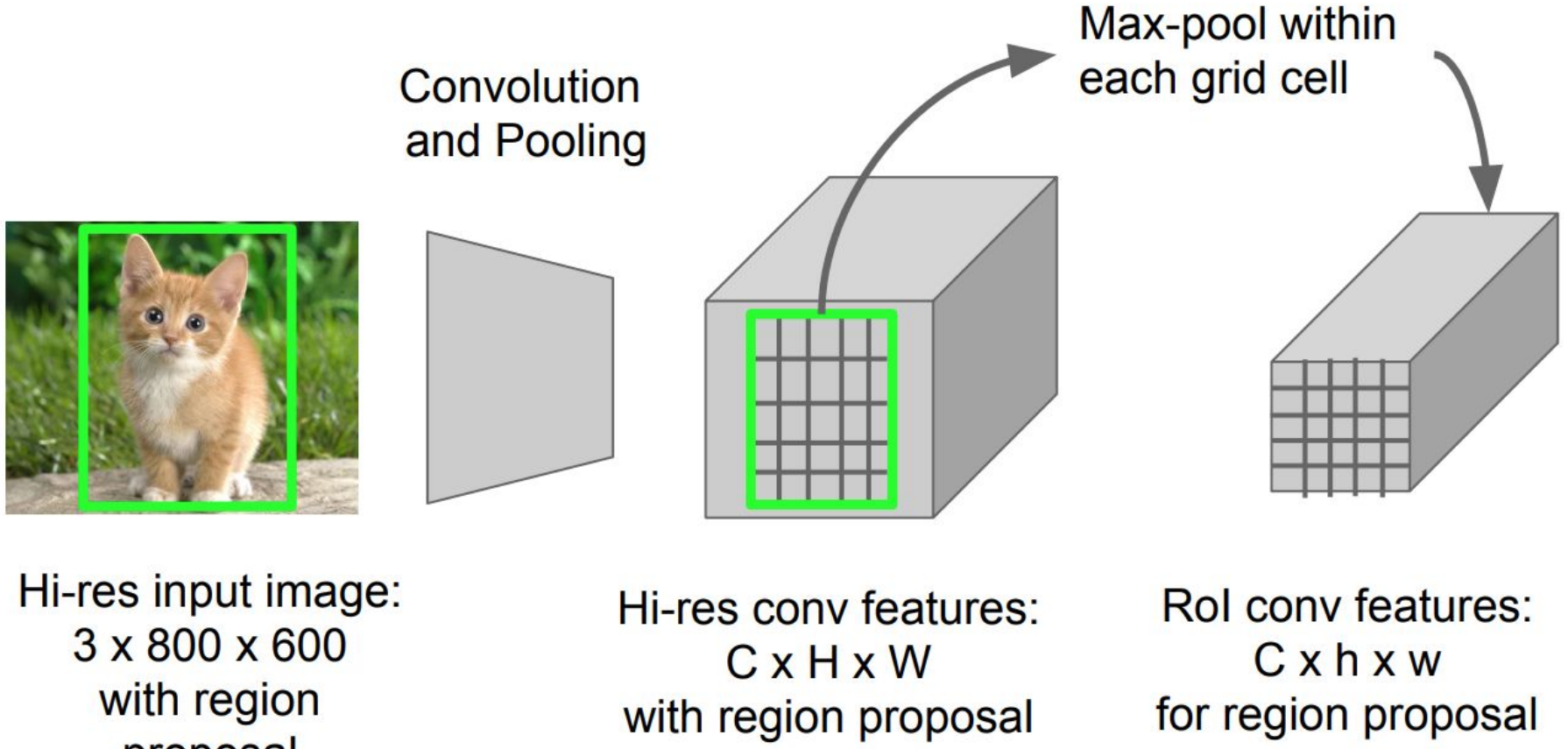
Faster R-CNN: RoI Pooling



http://cs231n.stanford.edu/slides/2016/winter1516_lecture8.pdf

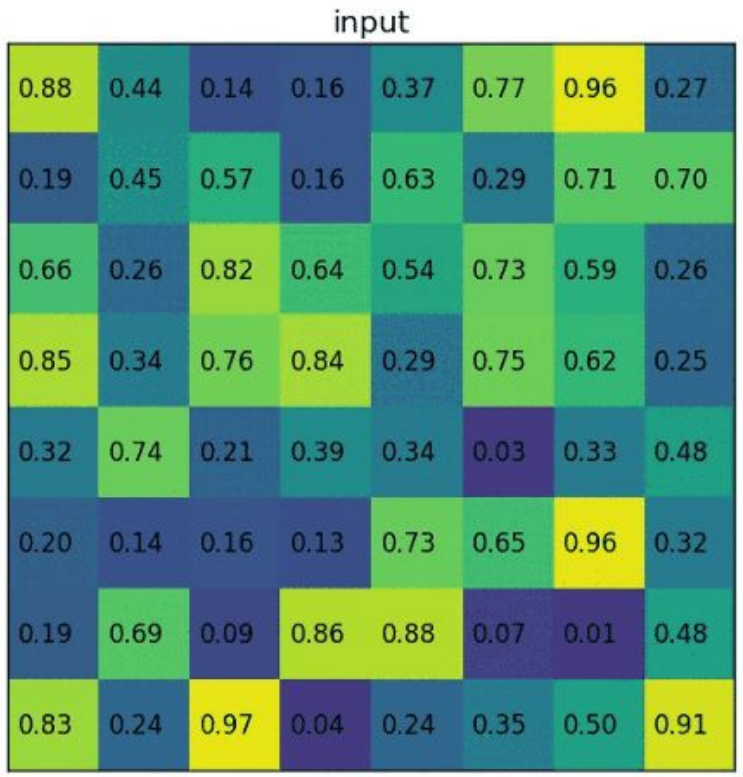
Faster R-CNN: RoI Pooling

Divide RoI into grid (same number of grid cells) and pool features



http://cs231n.stanford.edu/slides/2016/winter1516_lecture8.pdf

Faster R-CNN: RoI Pooling

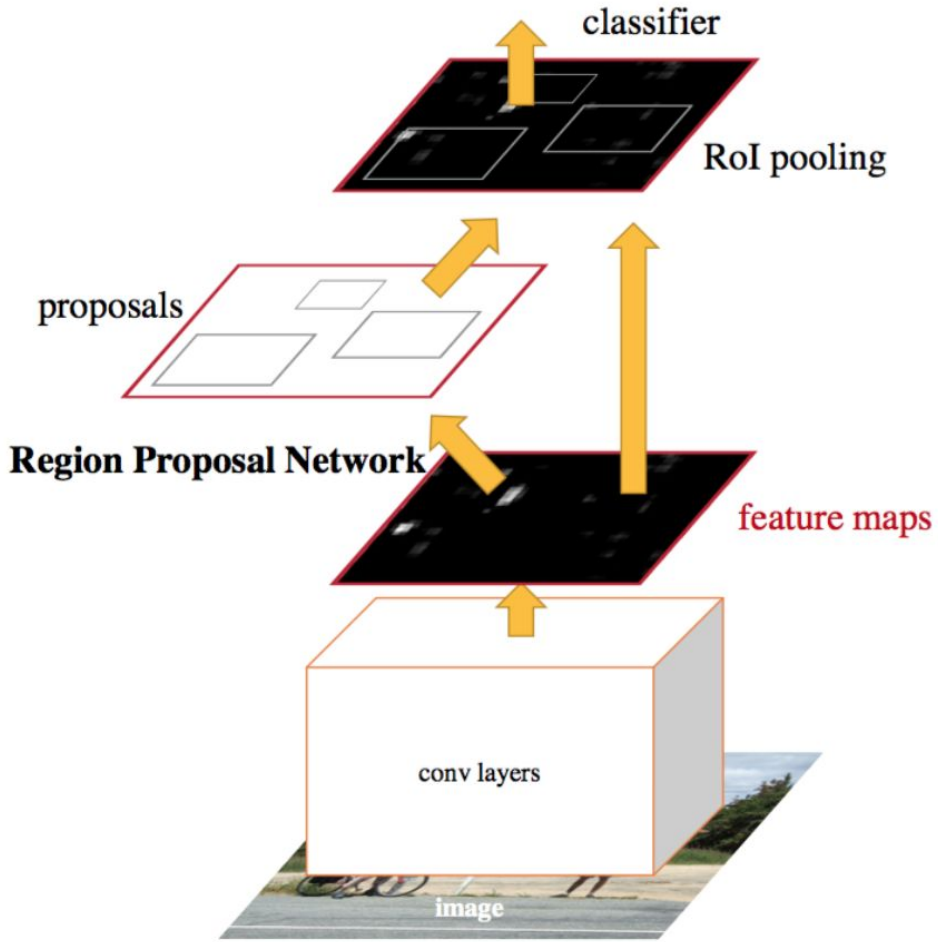


Divide RoI into grid

- always same number of grid cells
- Size of grid cells depends on RoI
- Apply max-pooling per cell

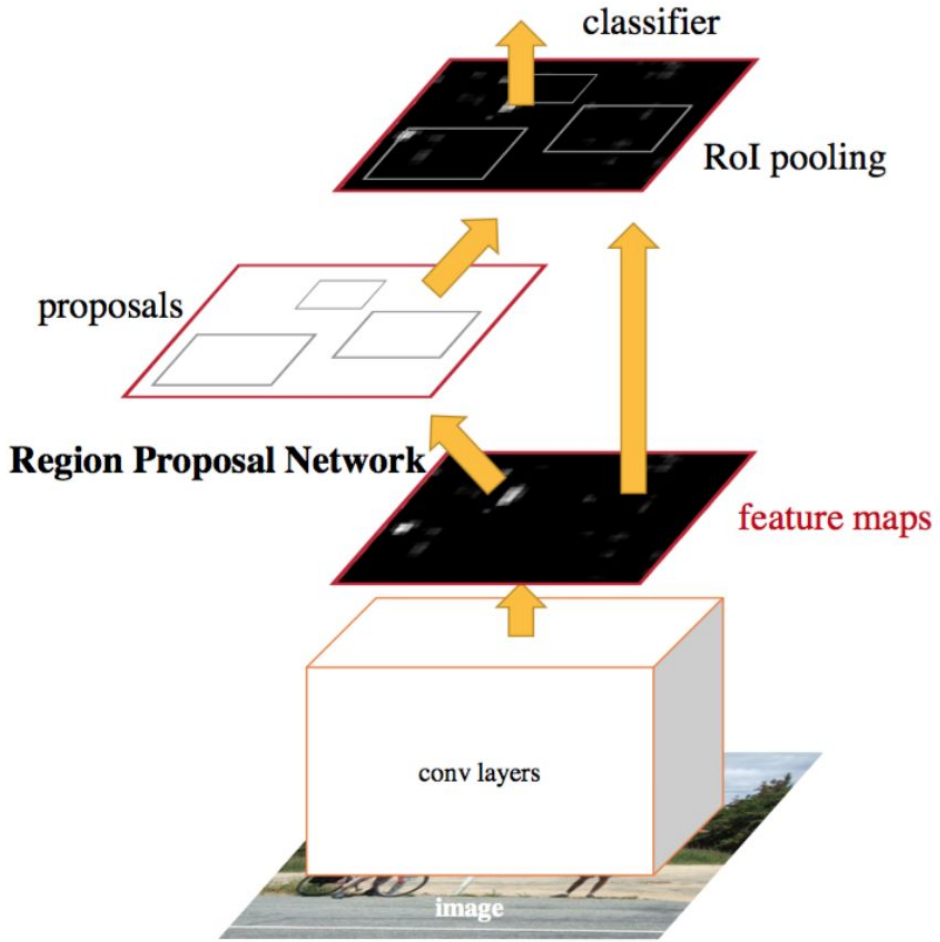
https://towardsdatascience.com/region-of-interest-pooling-f7c637f409af

Faster R-CNN: Architecture

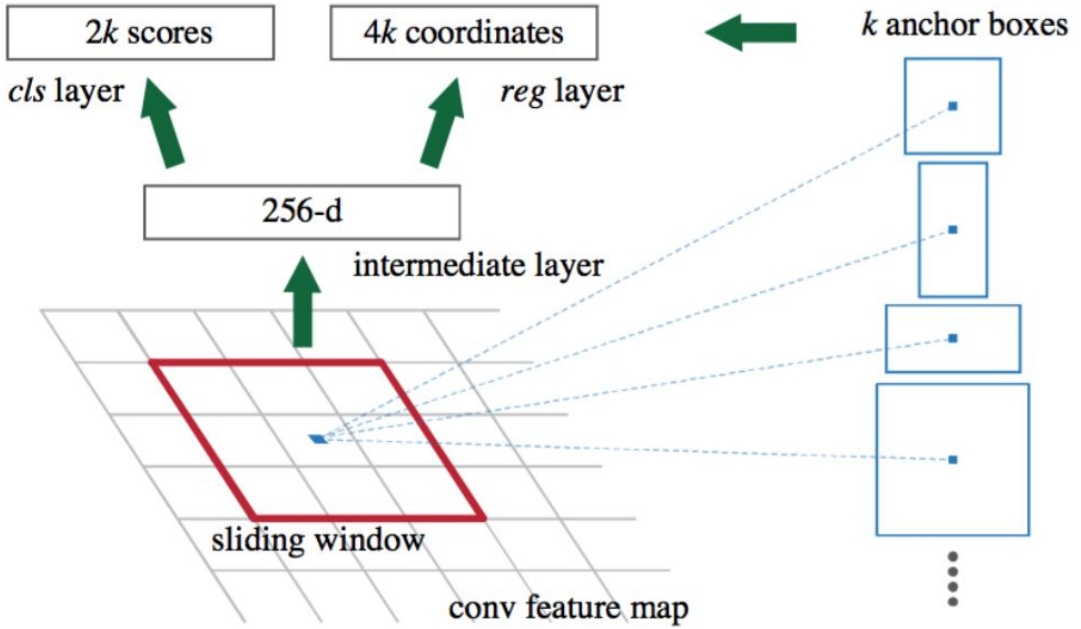


Ren et al., *NeurIPS* 2015

Faster R-CNN: Architecture

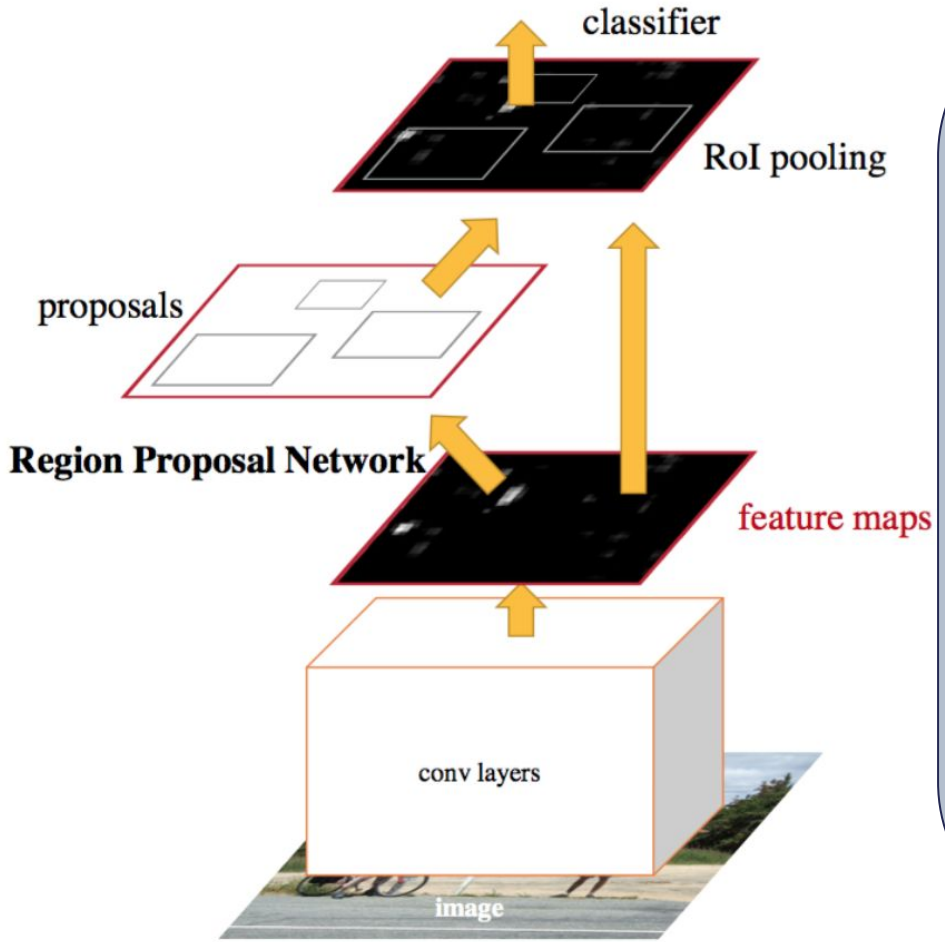


Output of the Region Proposal Network



Ren et al., *NeurIPS* 2015

Faster R-CNN: Loss function



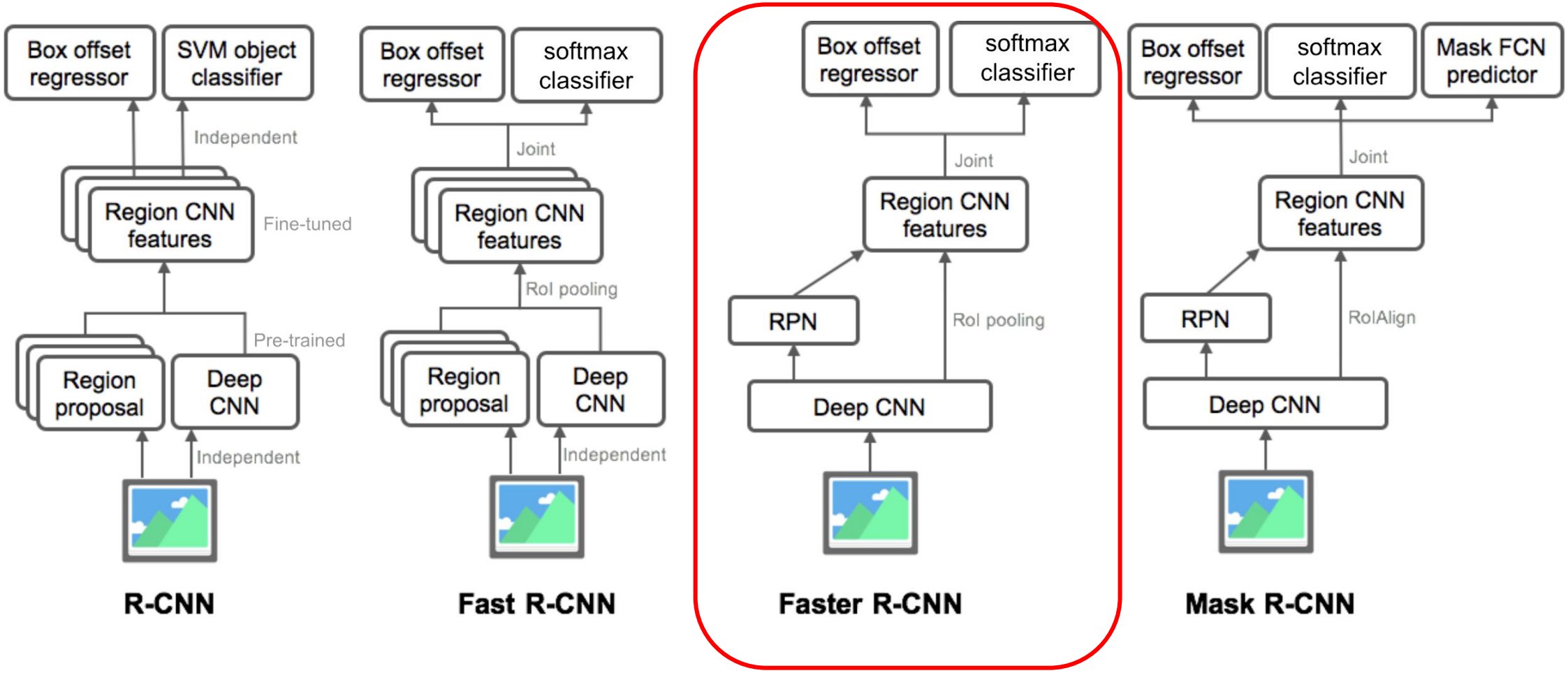
$$L(\{p_i\},\{t_i\}) = \frac{1}{N_{cls}}\sum_i L_{cls}(p_i, p_i^*) + \lambda\frac{1}{N_{reg}}\sum_i p_i^* L_{reg}(t_i, t_i^*).$$

i: number of region proposals

Ren et al., *NeurIPS* 2015

https://lilianweng.github.io/lil-log/2017/12/31/object-recognition-for-dummies-part-3.html

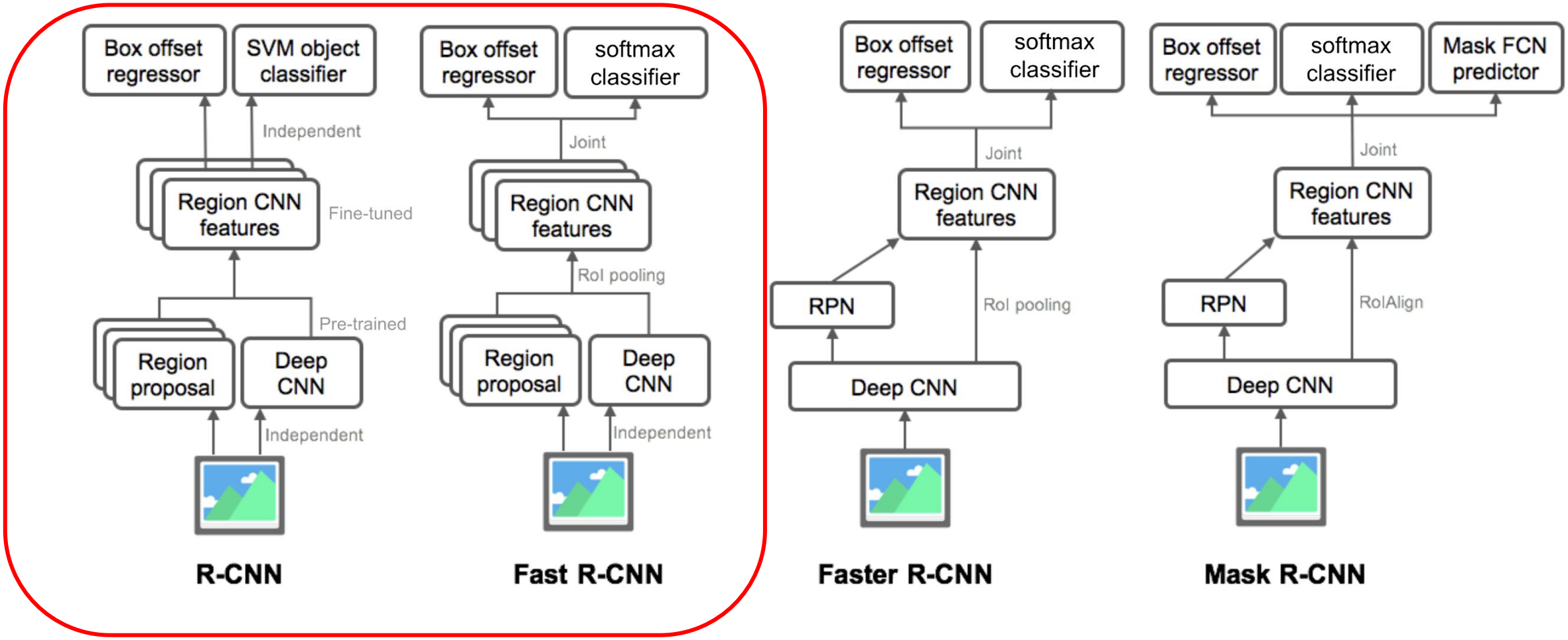
R-CNN Model family



What we just covered.

https://lilianweng.github.io/lil-log/2017/12/31/object-recognition-for-dummies-part-3.html

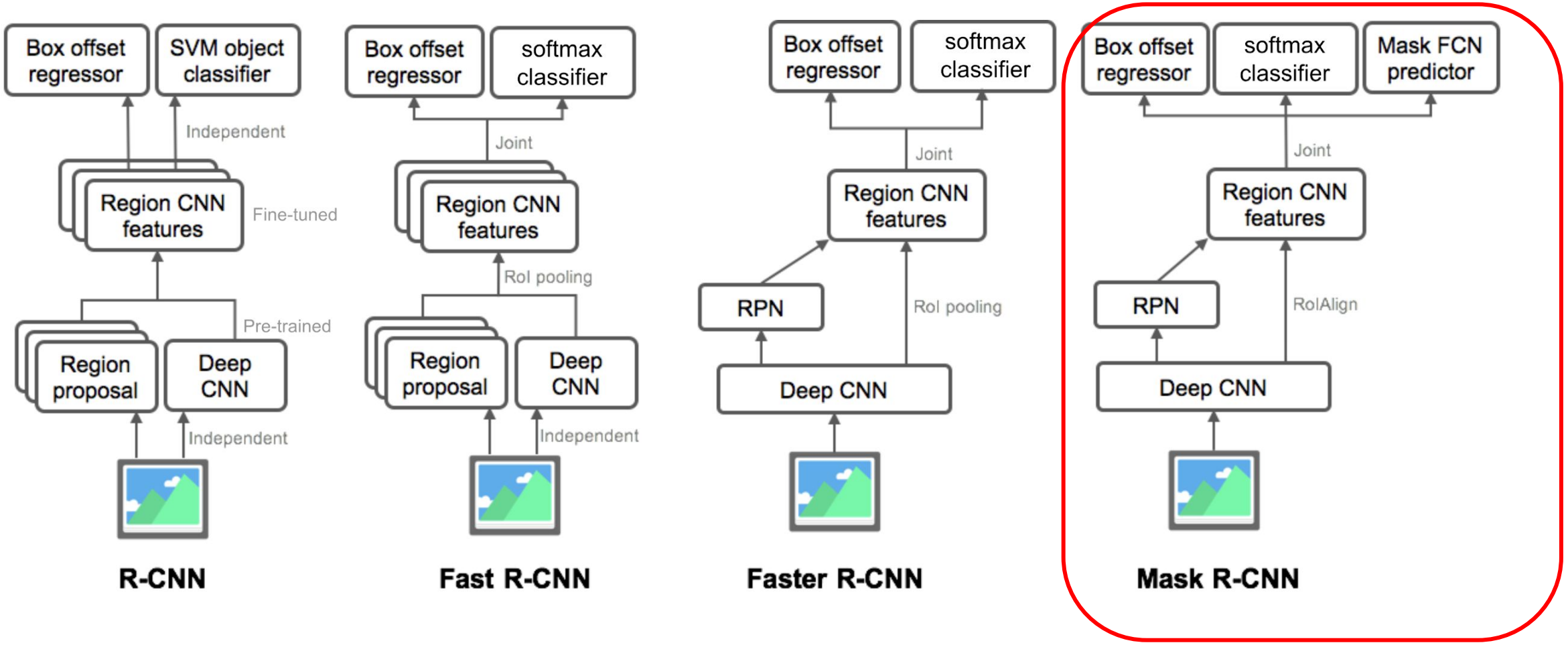
R-CNN Model family



Earlier versions, slower and worse performance

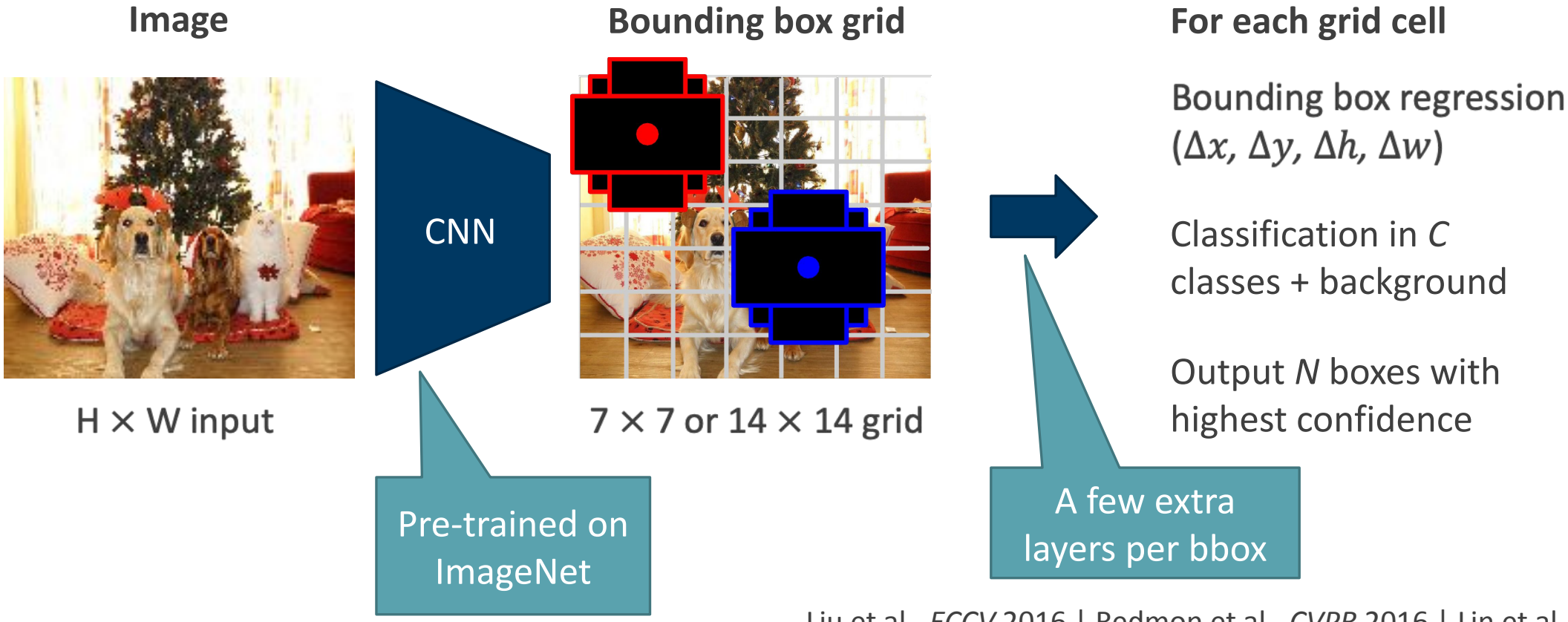
https://lilianweng.github.io/lil-log/2017/12/31/object-recognition-for-dummies-part-3.html

R-CNN Model family



Extension to instance segmentation

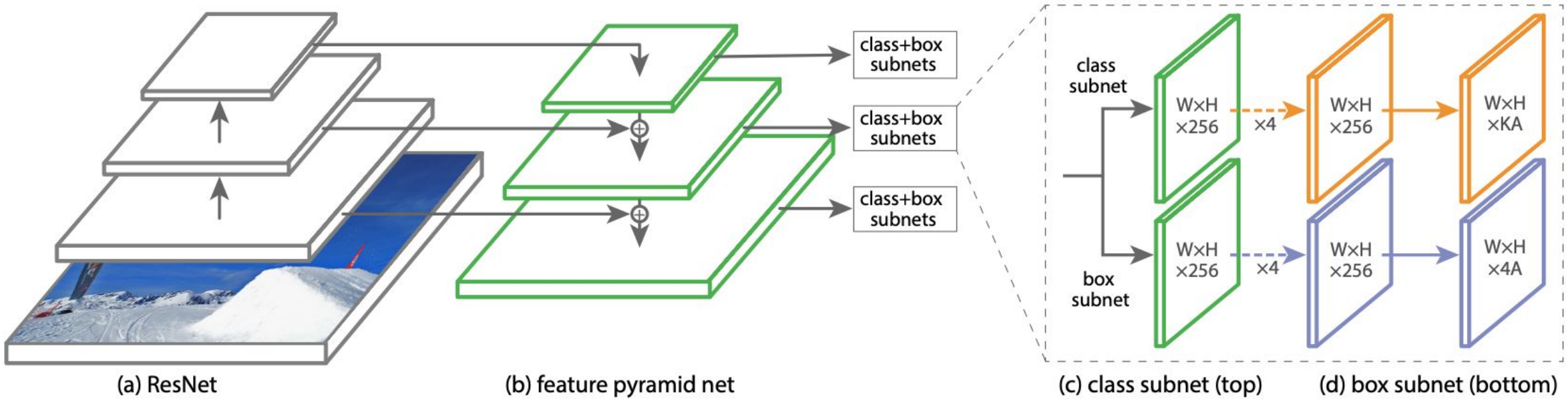
Single-stage detectors: SSD, YOLO, RetinaNet



Liu et al., *ECCV* 2016 | Redmon et al., *CVPR* 2016 | Lin et al., *CVPR* 2017

RetinaNet

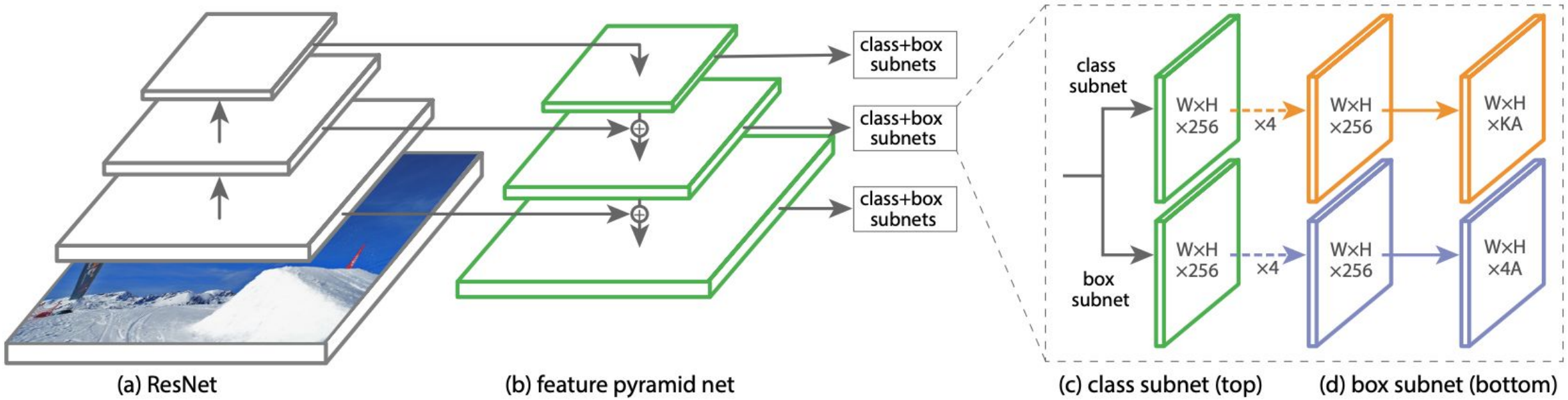
https://arxiv.org/abs/1708.02002



Region Proposals: fixed grid of bounding boxes for each level in feature pyramid

RetinaNet

https://arxiv.org/abs/1708.02002

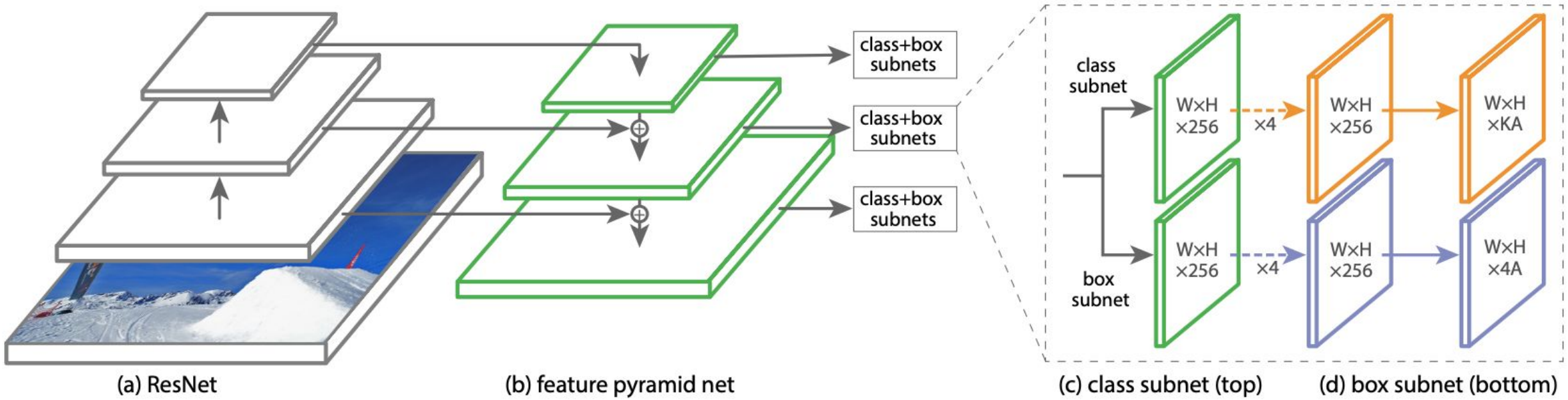


Training:

- Match bounding boxes to GT objects (overlap: 0.5)
- Classification net predicts scores for K classes (focal loss!)
- Box net regresses bounding box position

RetinaNet

https://arxiv.org/abs/1708.02002



Inference:

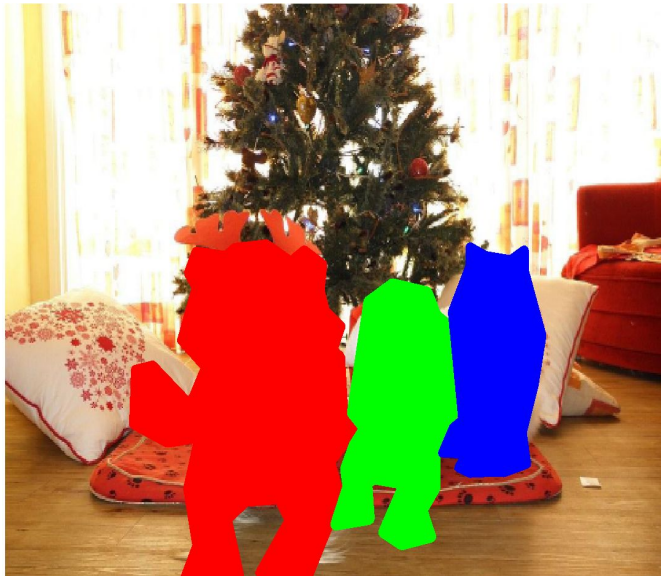
- Apply to image
- Merge predicted boxes from all levels
- Apply non-maximum suppression

Instance Segmentation

Instance segmentation

Goal:
Segment individual objects in the image
(= 1 mask per instance)

Main problem:
How many objects are there?



Dog, Dog, Cat

Instance segmentation

Representation:

- Background has pixel value 0
- each instance has a unique id, all pixels belonging to instance have that id

Instance segmentation

Representation:

- Background has pixel value 0
- each instance has a unique id, all pixels belonging to instance have that id

```
00000000000000000000
00000001111111111100000
00000011111111111100000
00000011111111122222222
00000000000000000222222
00333333300000000022200
00003333333300000002000
00000033330000000000000
00000003330000000000000
```

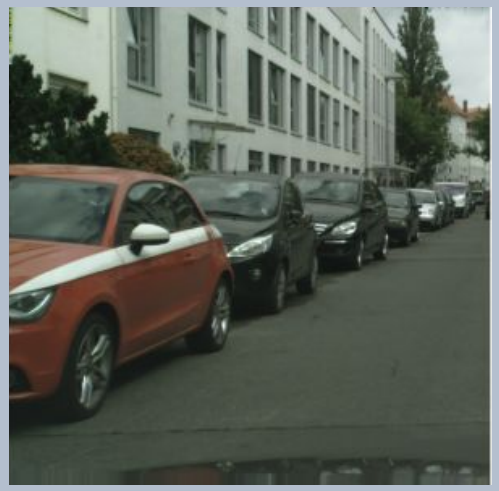
Instance segmentation

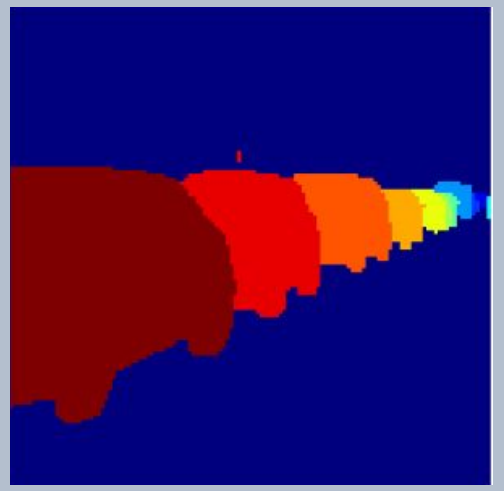
Representation:

- Background has pixel value 0
- each instance has a unique id, all pixels belonging to instance have that id

```
00000000000000000000
00000001111111111100000
00000011111111111100000
00000011111111122222222
00000000000000000222222
00333333300000000022200
00003333333300000002000
00000033330000000000000
00000003330000000000000
```

Visualize with random colors





Instance segmentation

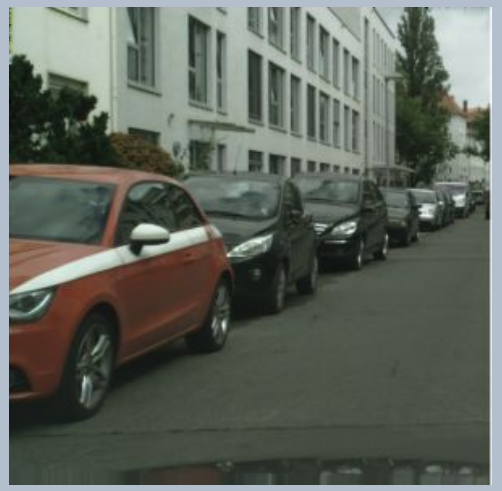
Other instance representations: boundaries, polygons, meshes (3d), …

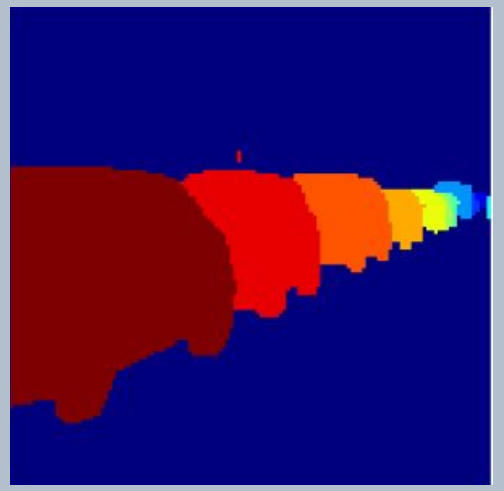
Representation:

- Background has pixel value 0
- each instance has a unique id, all pixels belonging to instance have that id

```
00000000000000000000
00000001111111111100000
00000011111111111100000
00000011111111122222222
00000000000000000222222
00333333300000000022200
00003333333300000002000
00000033330000000000000
00000003330000000000000
```

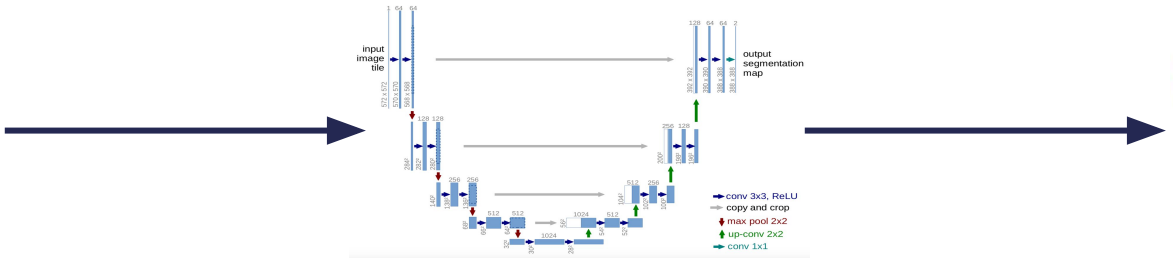
Visualize with random colors

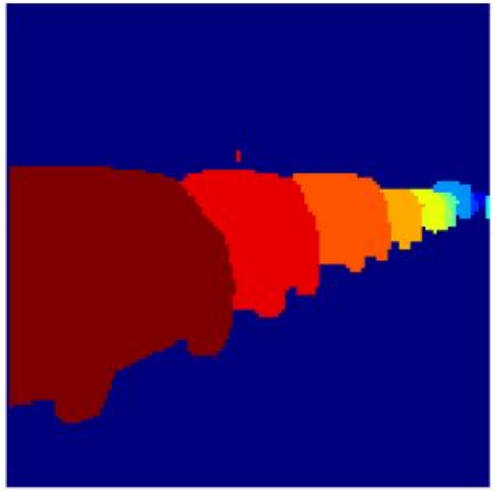




Instance segmentation

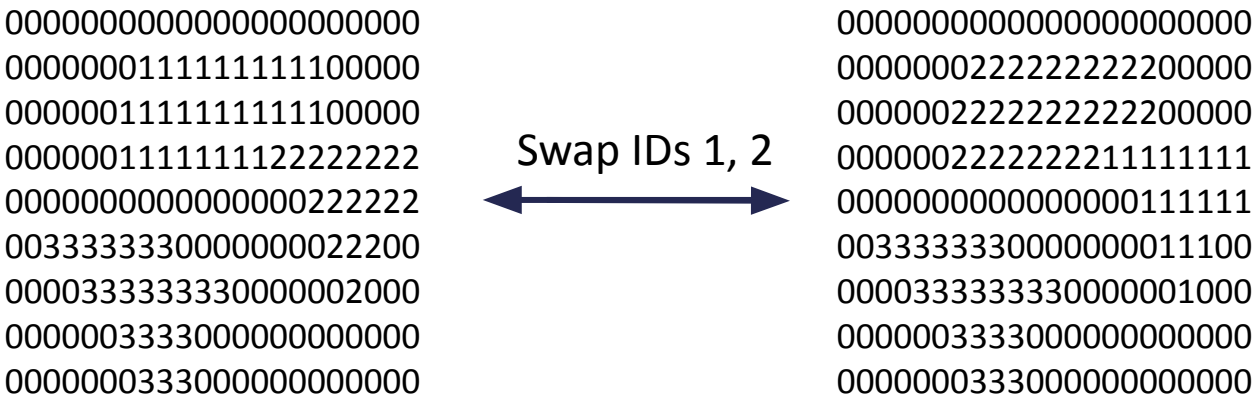
Predict instances directly?





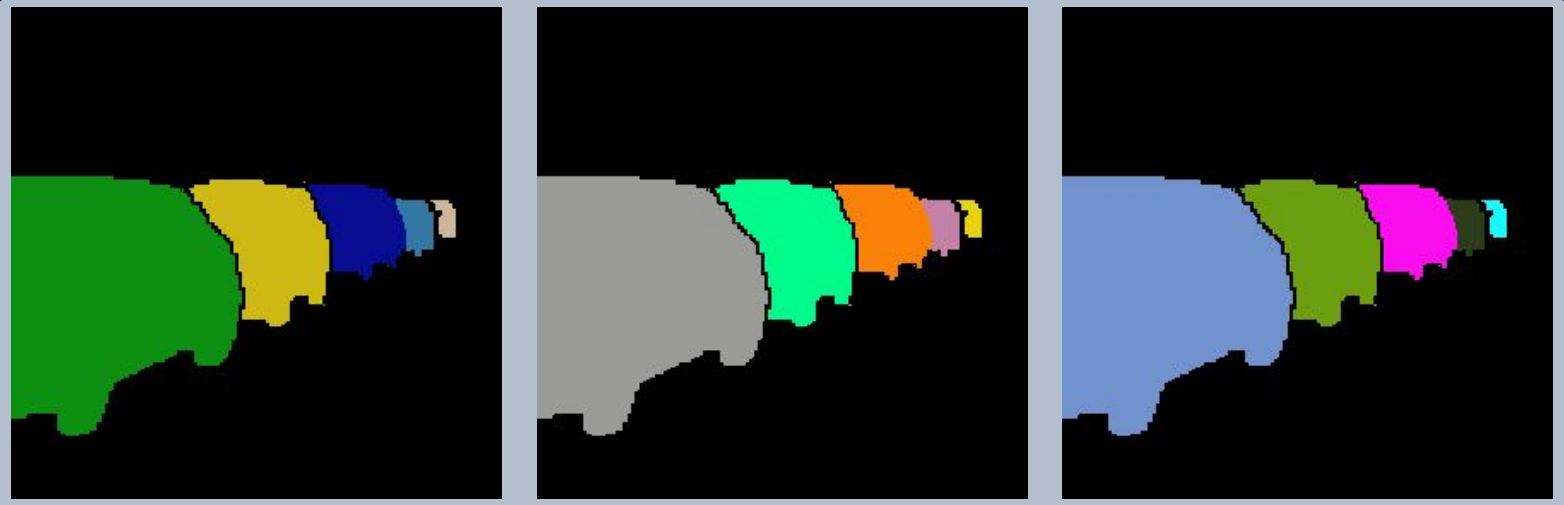
Instance segmentation

Problem: instance segmentation is invariant under relabeling:
e.g. swapping instance labels *1* and *2* does not change the solution



Instance segmentation

Problem: instance segmentation is invariant under relabeling:
e.g. swapping instance labels *1* and *2* does not change the solution

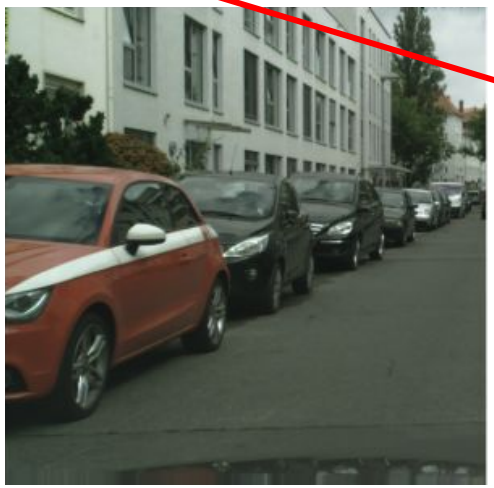


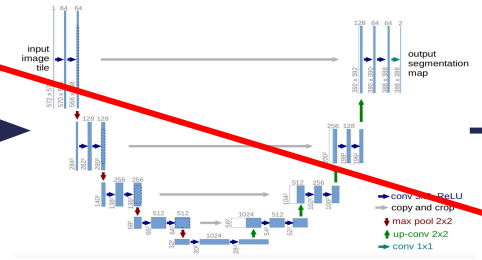
Equivalent instance segmentations!

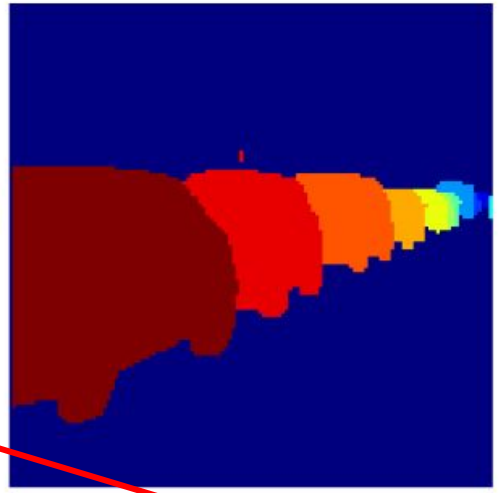
Instance segmentation

*On-going research: differentiable instance segmentation

Predict instances directly?
Not possible due to relabeling invariance, can't formulate a differentiable loss*







Instance segmentation

Solutions:

- *Proposal-based* instance segmentation: perform object detection, predict one instance mask per bounding box

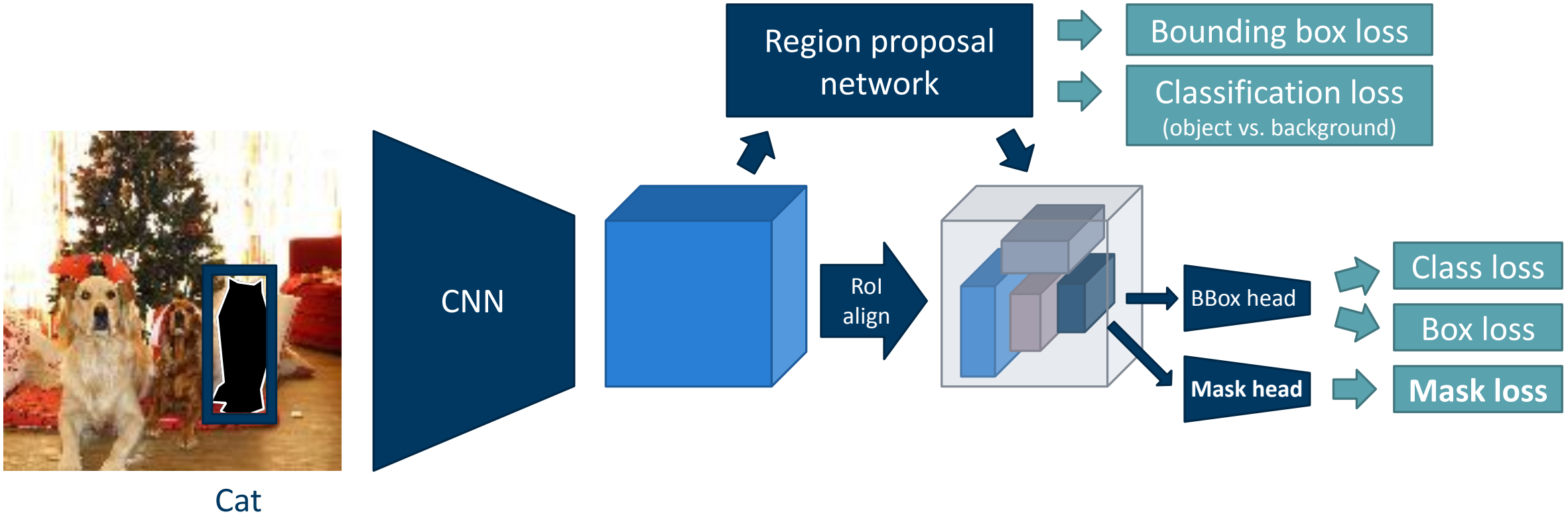
Instance segmentation

Solutions:

- *Proposal-based* instance segmentation: perform object detection, predict one instance mask per bounding box

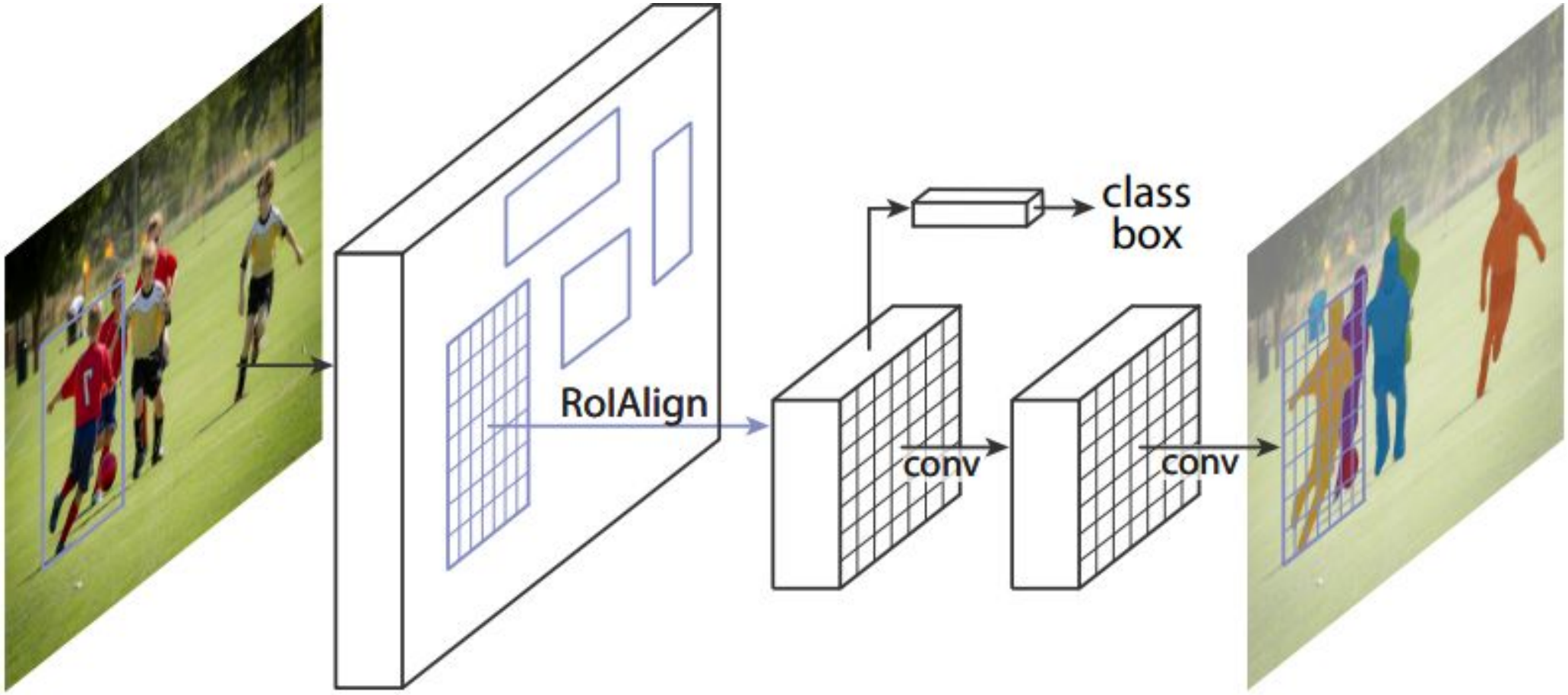
- *Proposal-free* instance segmentation: predict intermediate representation (differentiable loss!), find instances in post-processing

Proposal-based: Mask R-CNN



He et al., *CVPR* 2017

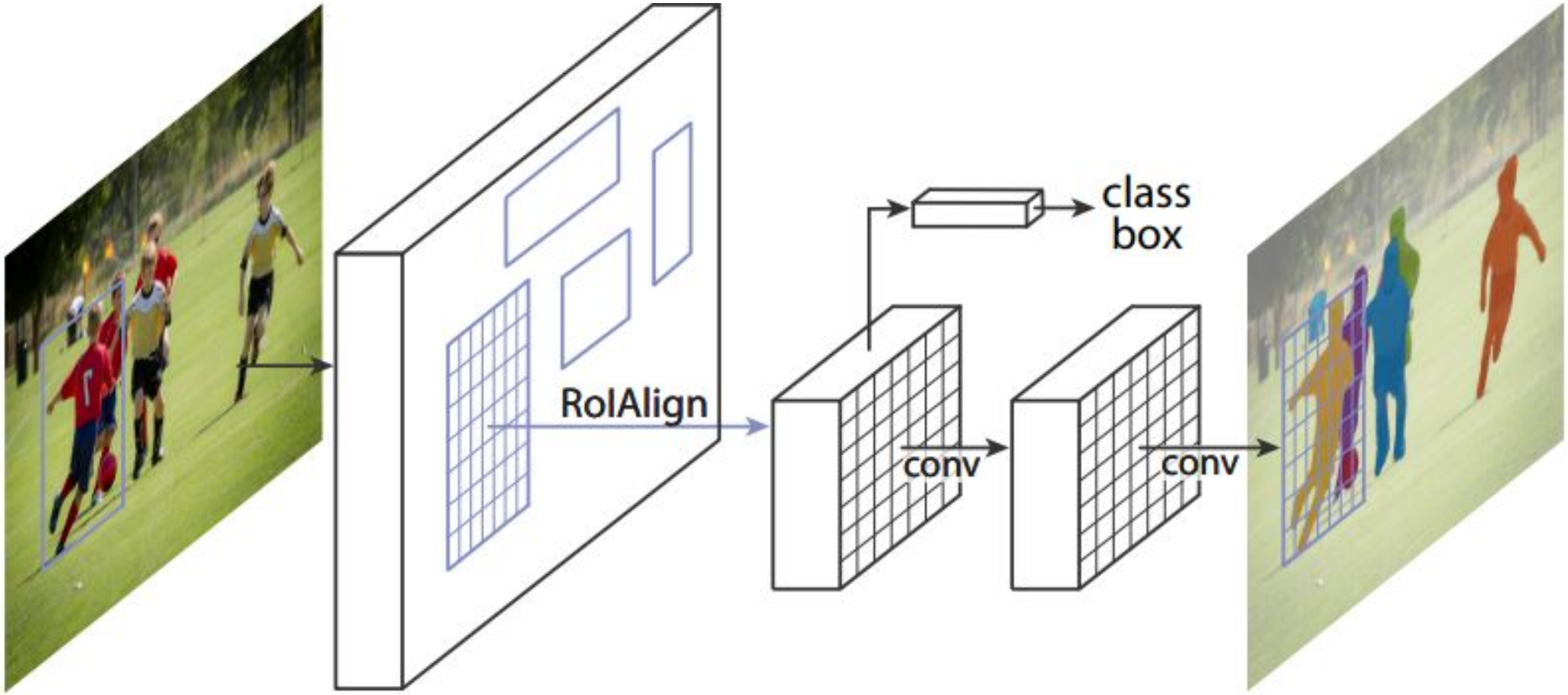
Mask R-CNN



He et al., *CVPR* 2017

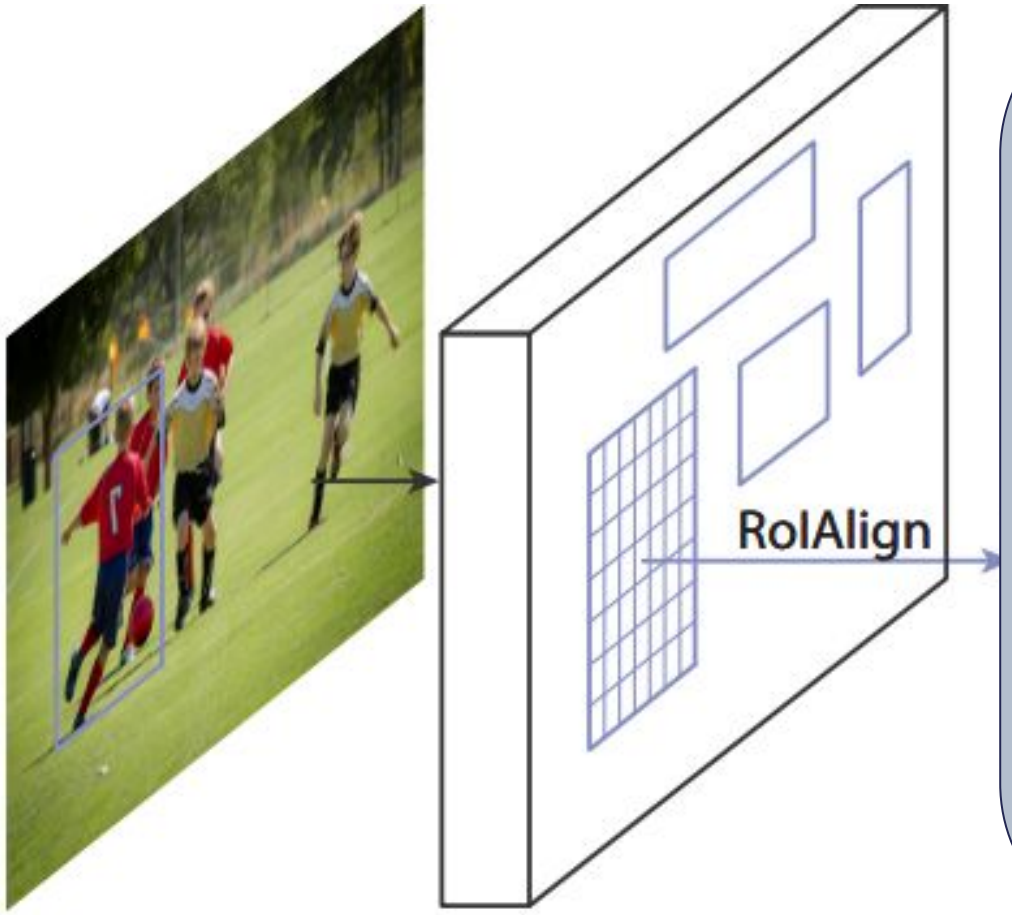
Mask R-CNN

RoIAlign vs. RoiPool:
bilinear interpolation instead of max pooling
-> features are better aligned with the image;
important for correct masks



He et al., *CVPR* 2017

Mask R-CNN

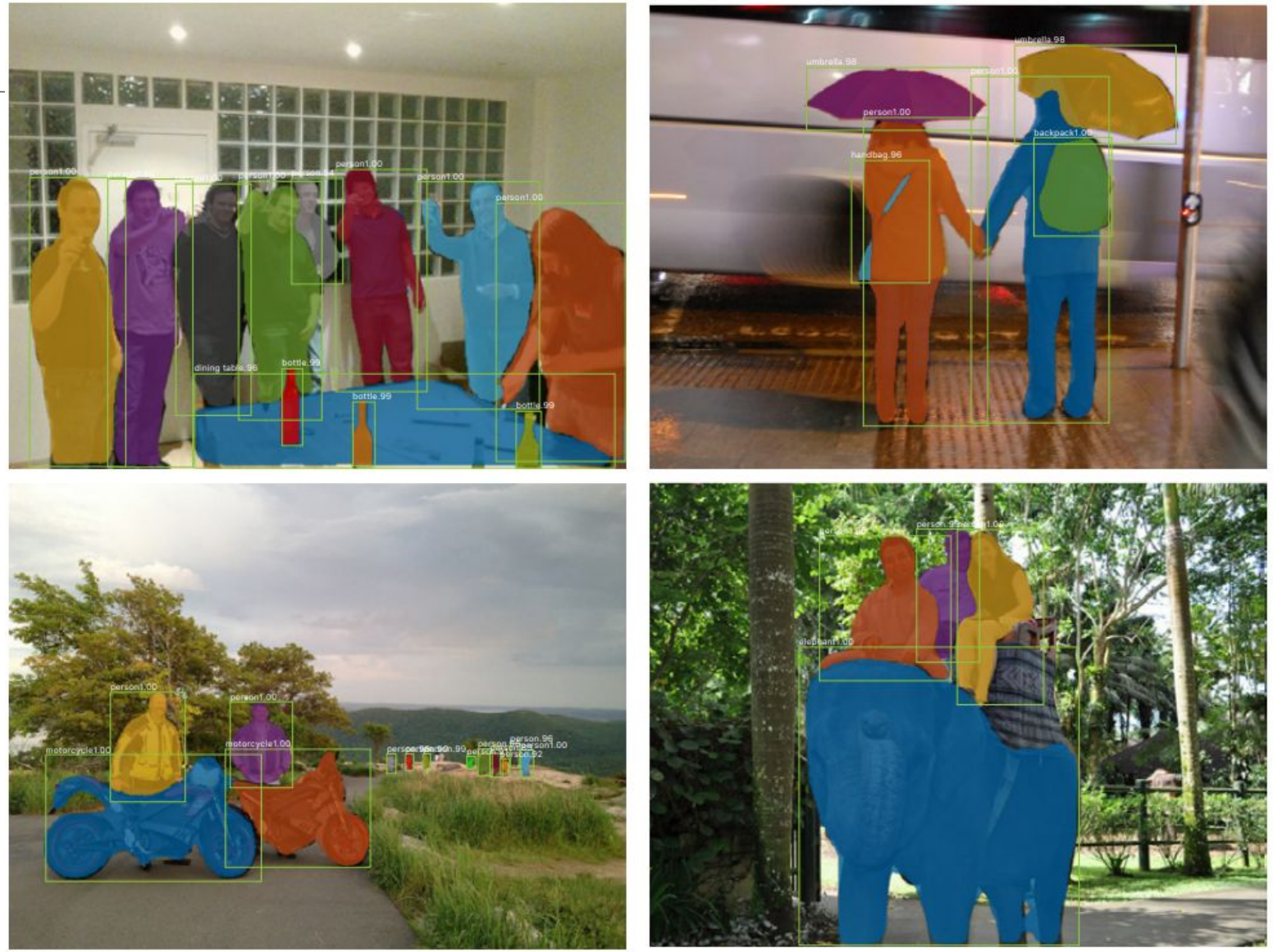


$$L = L_{cls} + L_{box} + L_{mask}$$

He et al., *CVPR* 2017

Mask R-CNN

One of the most common instance segmentation methods for natural images.

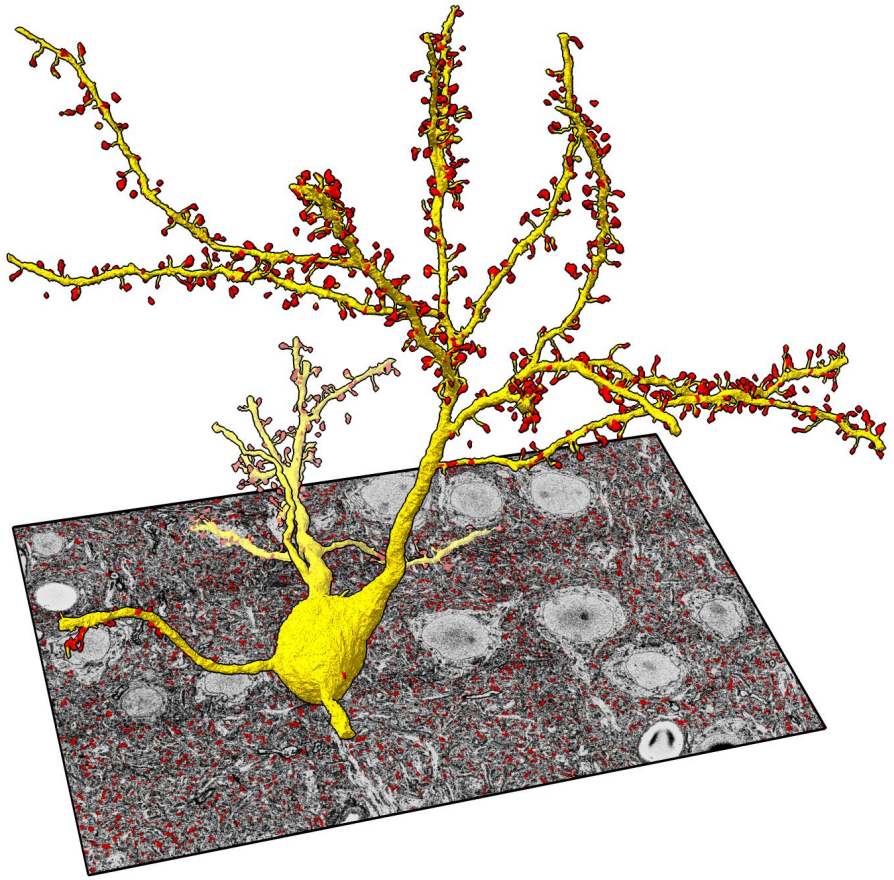


Mask R-CNN

https://www.rbvi.ucsf.edu/chimerax/data/layer4-june2020/mousebrain.html

One of the most common instance segmentation methods for natural images.

But: downsides for large objects with complex shapes:
Bounding box much bigger than mask!



Proposal-free instance segmentation

Strategy: predict intermediate representation, find instances in post-processing

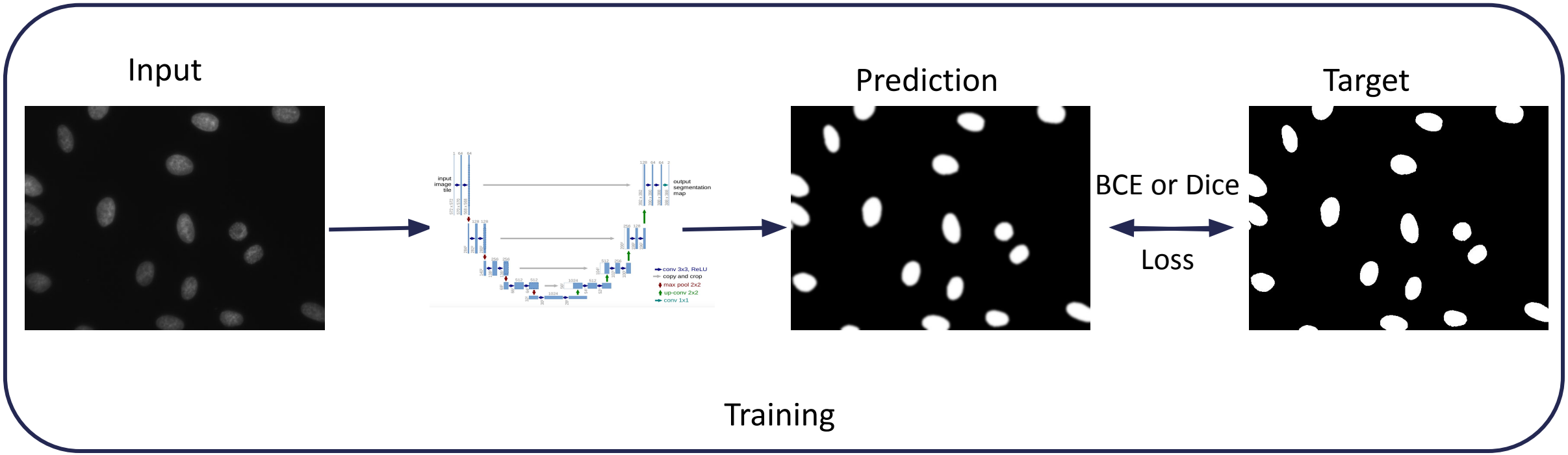
- Intermediate representation needs a suitable loss function (differentiable)
- Post-processing is not learned -> **not** end-to-end
- Example representations and post-processing
  - Foreground predictions (semantic segmentation) + connected components
  - Boundary predictions (semantic segmentation) + agglomerative clustering
  - Pixel embeddings (metric learning) + density clustering

Proposal-free: foreground based

Predict foreground (semantic segmentation), apply *connected components*

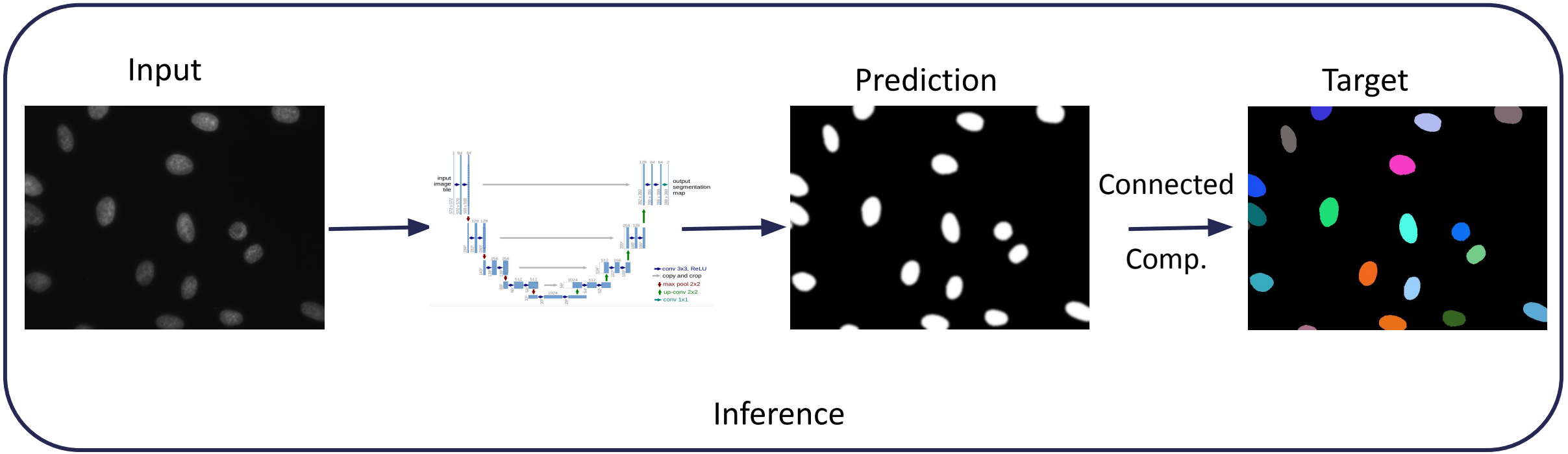
Proposal-free: foreground based

Predict foreground (semantic segmentation), apply *connected components*
Example: nucleus segmentation in fluorescence microscopy



Proposal-free: foreground based

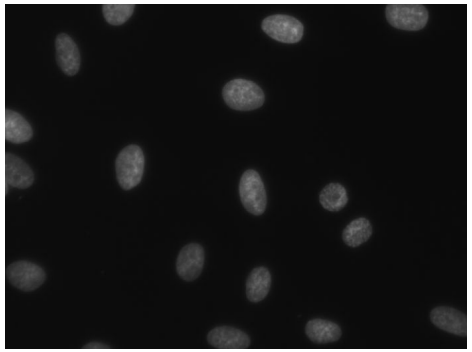
Predict foreground (semantic segmentation), apply *connected components*
Example: nucleus segmentation in fluorescence microscopy

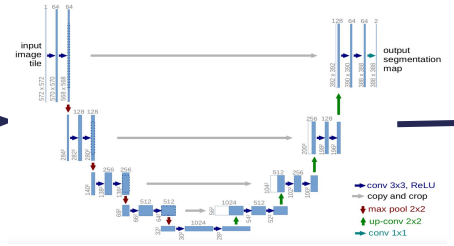


Proposal-free: foreground based

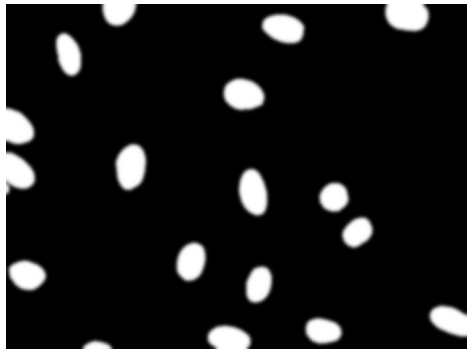
Predict foreground (semantic segmentation), apply *connected components*
Example: nucleus segmentation in fluorescence microscopy

Input



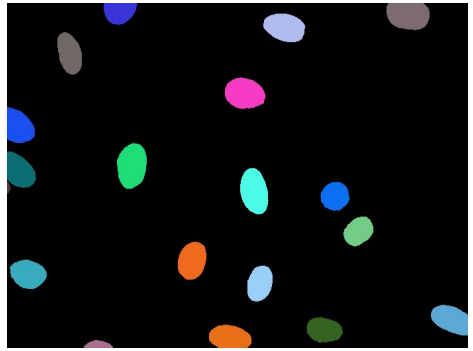


Prediction



Connected Comp.

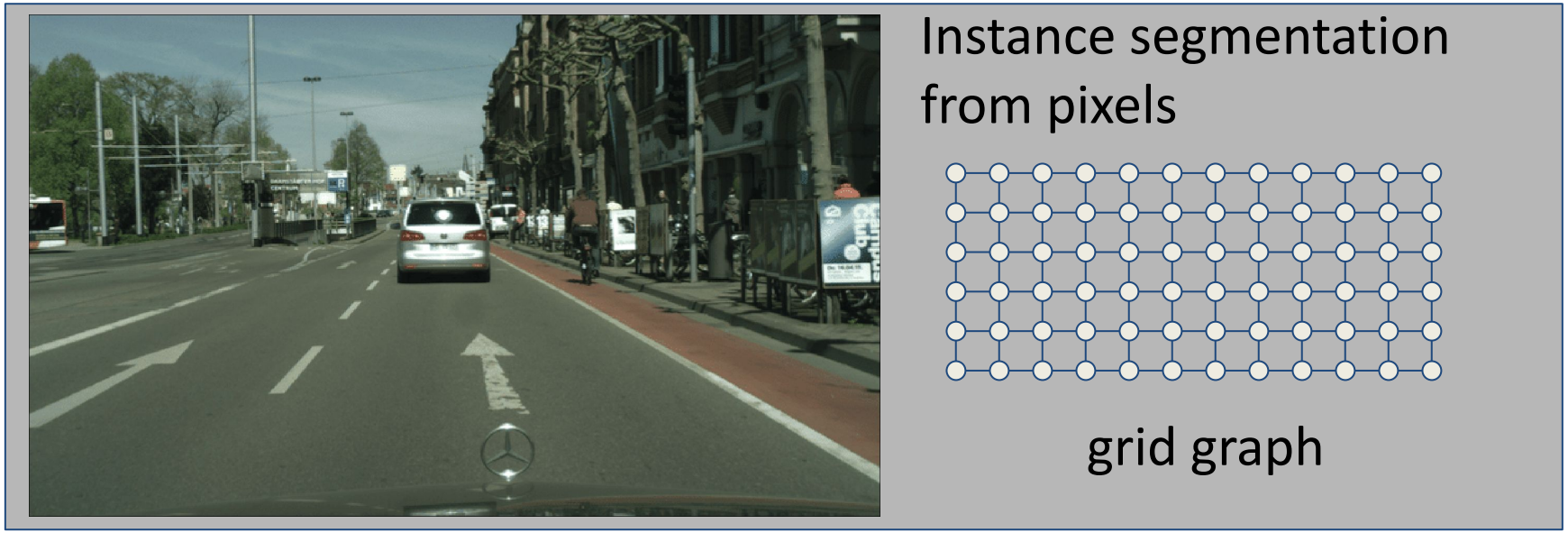
Target



Problem: touching objects!

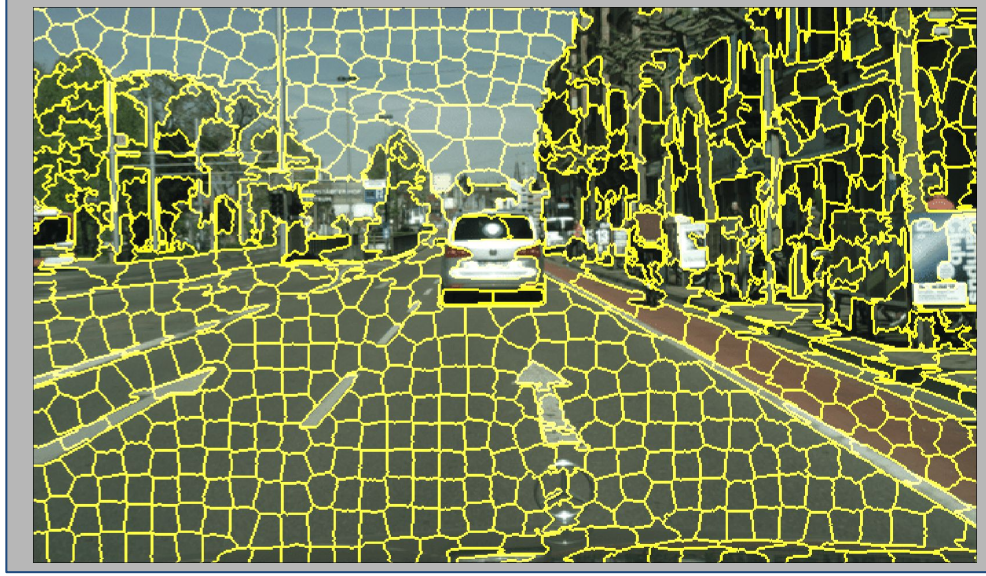
Proposal-free: boundary based

Predict boundaries (semantic segmentation),
find instances by (graph-based) agglomerative clustering

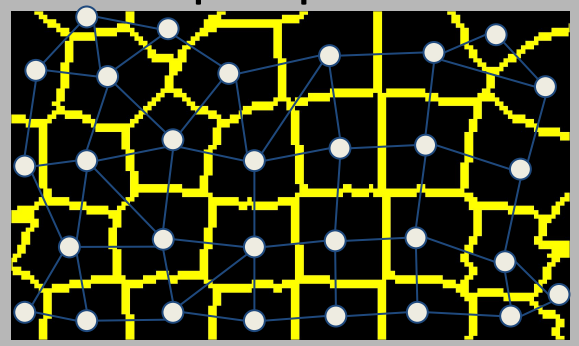


Proposal-free: boundary based

Predict boundaries (semantic segmentation),
find instances by (graph-based) agglomerative clustering



Instance segmentation
from super-pixels



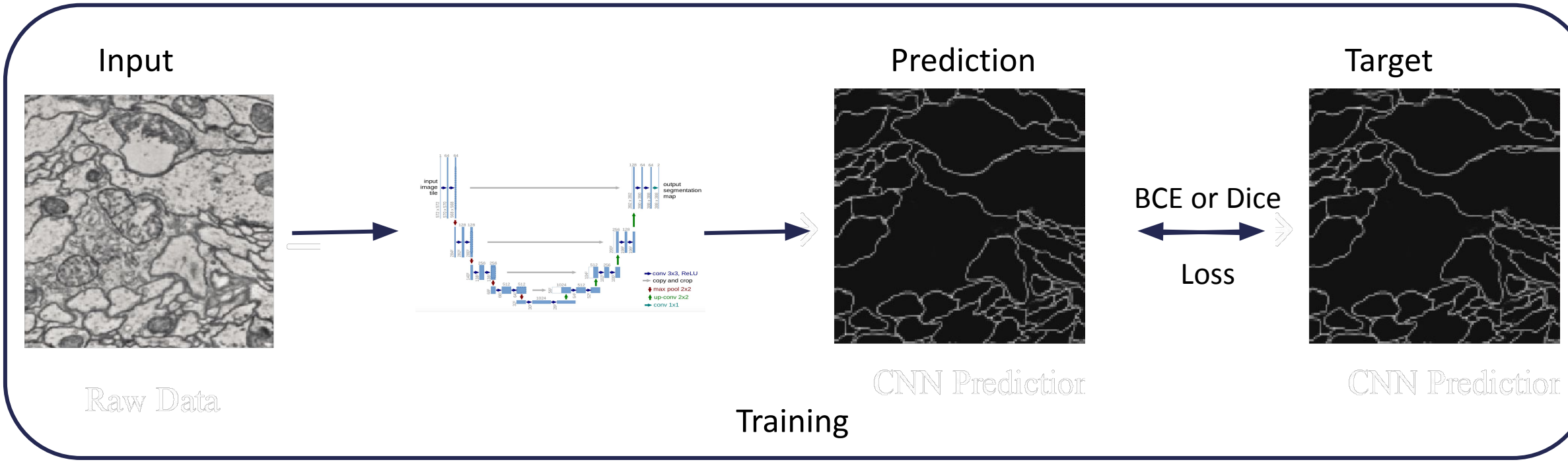
irregular graph

https://arxiv.org/abs/1906.11713

https://openaccess.thecvf.com/content_ECCV_2018/html/Steffen_Wolf_The_Mutex_Watershed_ECCV_2018_paper.html

Boundary based segmentation

Application: neuron segmentation in electron microscopy

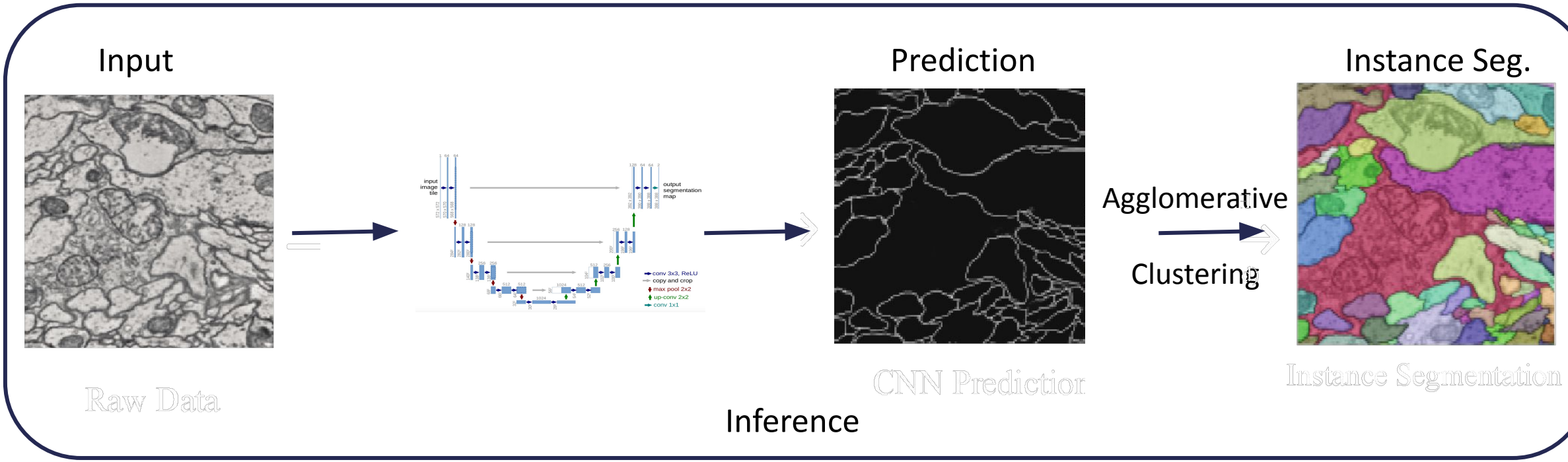


https://arxiv.org/abs/1906.11713

https://openaccess.thecvf.com/content_ECCV_2018/html/Steffen_Wolf_The_Mutex_Watershed_ECCV_2018_paper.html

Boundary based segmentation

Application: neuron segmentation in electron microscopy

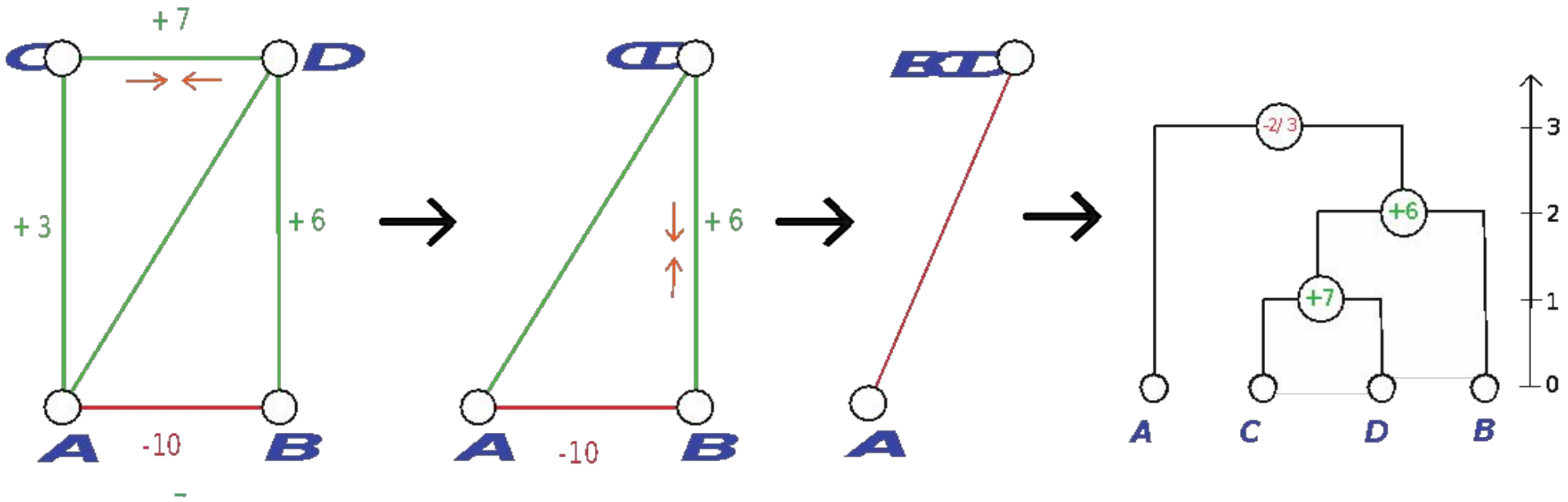


https://arxiv.org/abs/1906.11713

https://openaccess.thecvf.com/content_ECCV_2018/html/Steffen_Wolf_The_Mutex_Watershed_ECCV_2018_paper.html

Boundary based segmentation

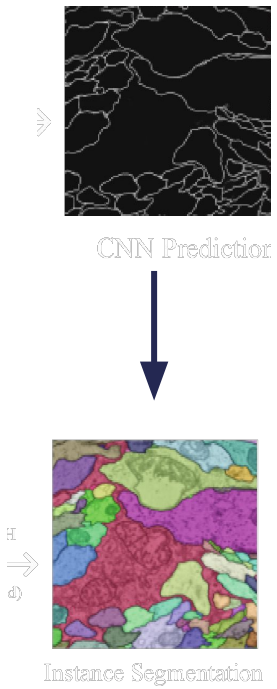
Agglomerative clustering: contract nodes in graph based on edge weights

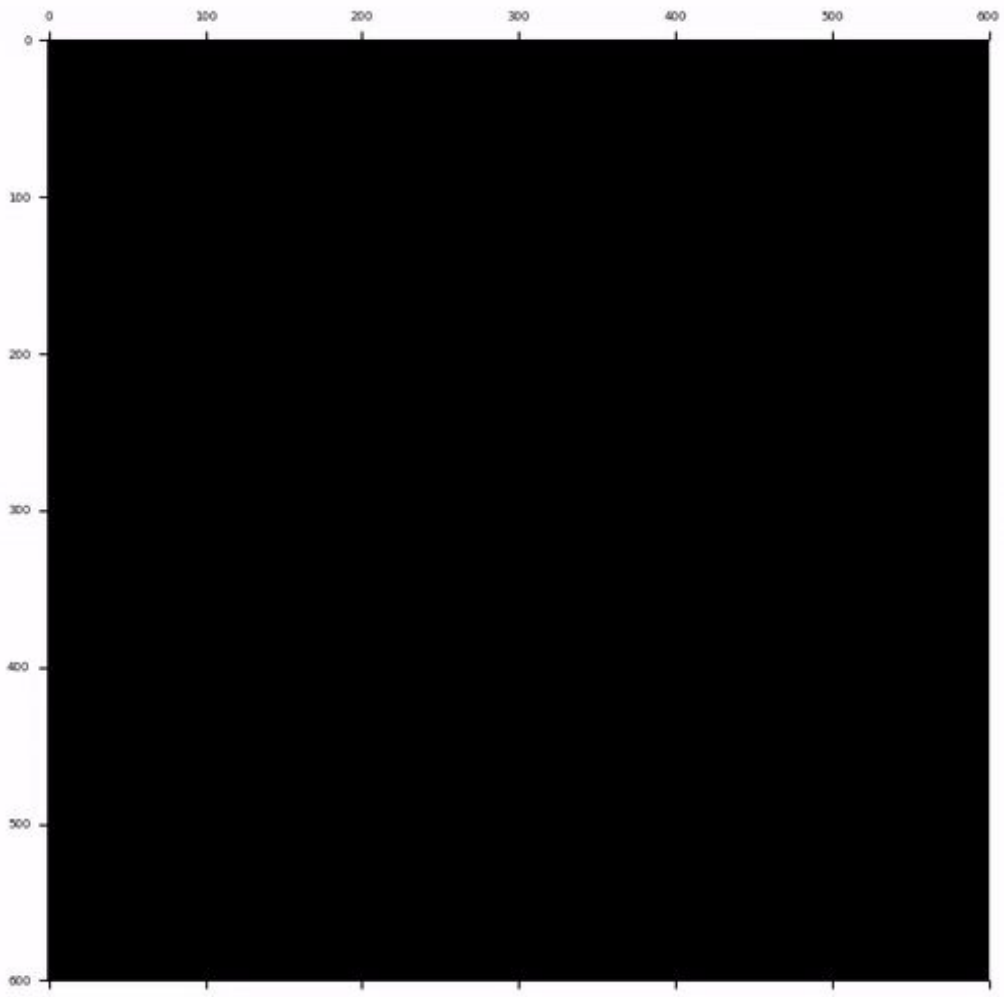


https://arxiv.org/abs/1906.11713

https://openaccess.thecvf.com/content_ECCV_2018/html/Steffen_Wolf_The_Mutex_Watershed_ECCV_2018_paper.html

Boundary based segmentation

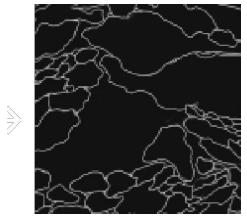




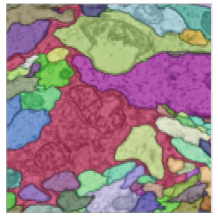
https://arxiv.org/abs/1906.11713

https://openaccess.thecvf.com/content_ECCV_2018/html/Steffen_Wolf_The_Mutex_Watershed_ECCV_2018_paper.html

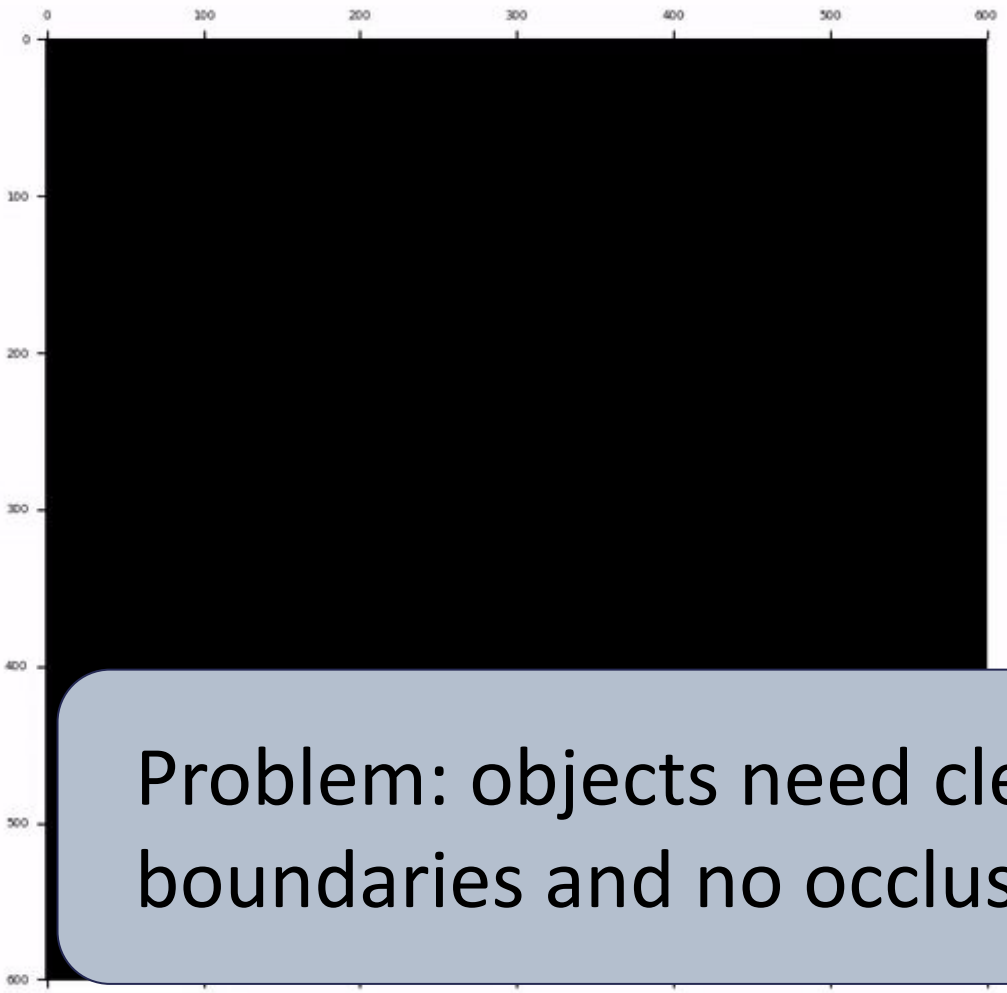
Boundary based segmentation



CNN Prediction



Instance Segmentation

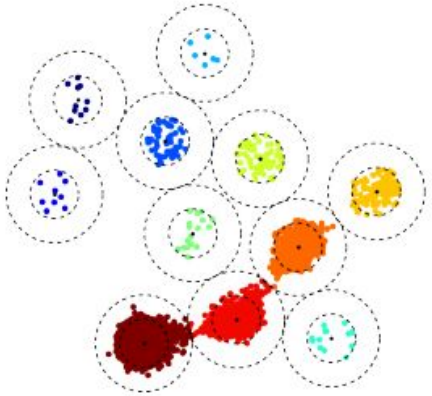


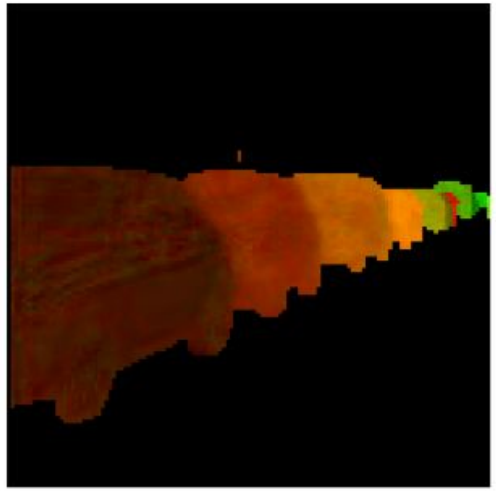
Problem: objects need clear boundaries and no occlusion!

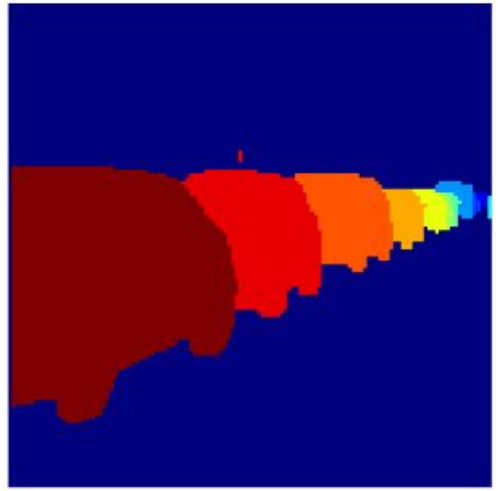
https://arxiv.org/abs/1708.02551

Proposal-free: embedding based

Predict per-pixel embeddings (metric learning), find instances by density clustering



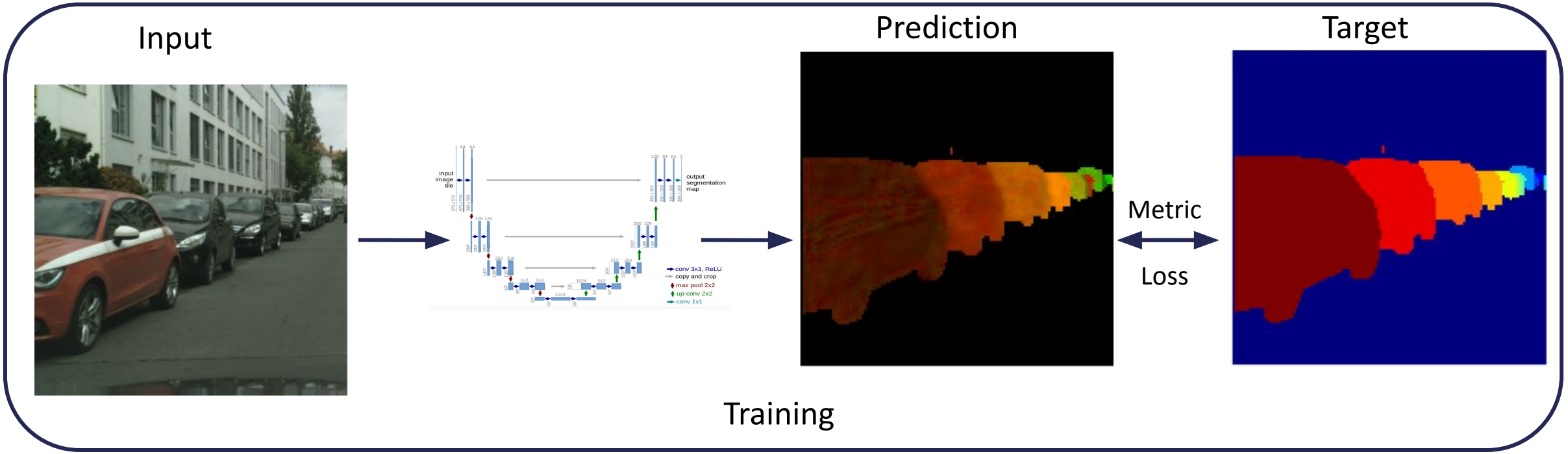




https://arxiv.org/abs/1708.02551

Embedding based segmentation

Example method: De Brabandere et al. Semantic Instance Segmentation with a Discriminative Loss Function

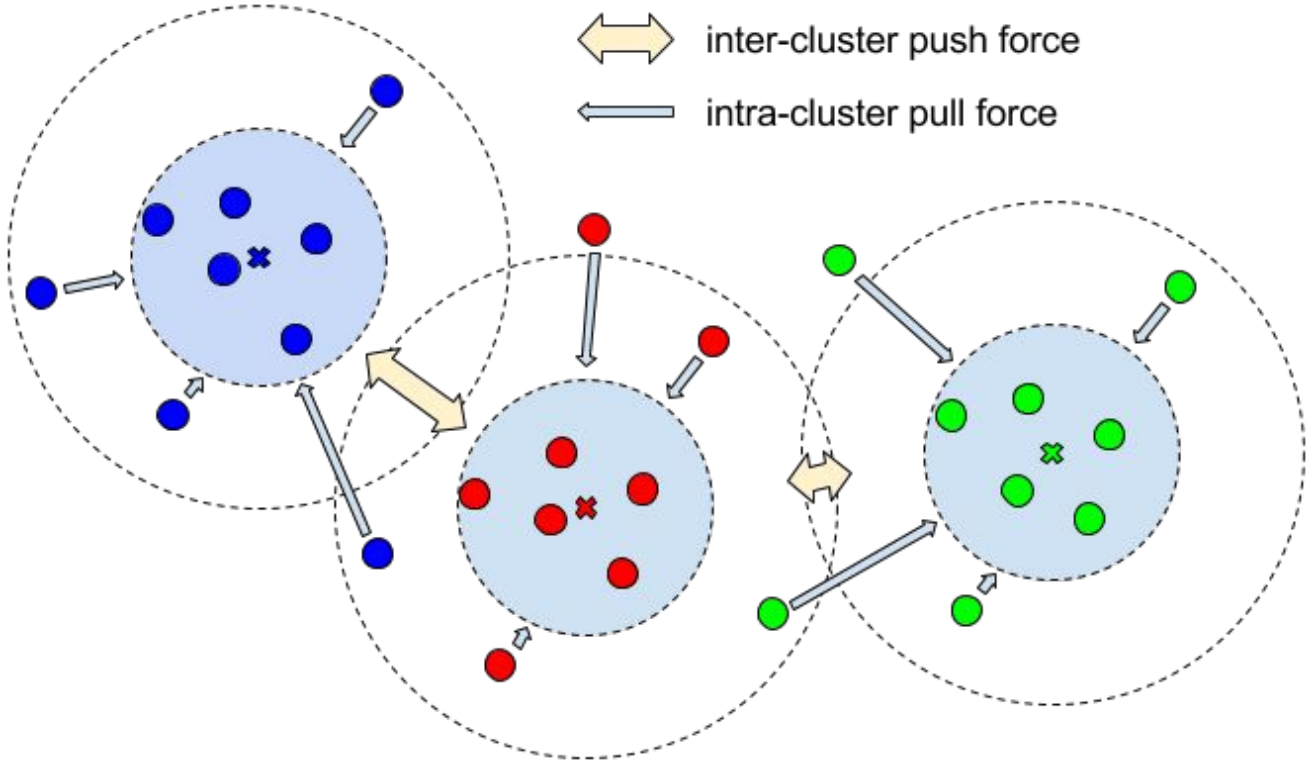


https://arxiv.org/abs/1708.02551

Embedding based segmentation

Loss function:

- attract pixels that belong to the same instance
- repel pixels belonging to different instances



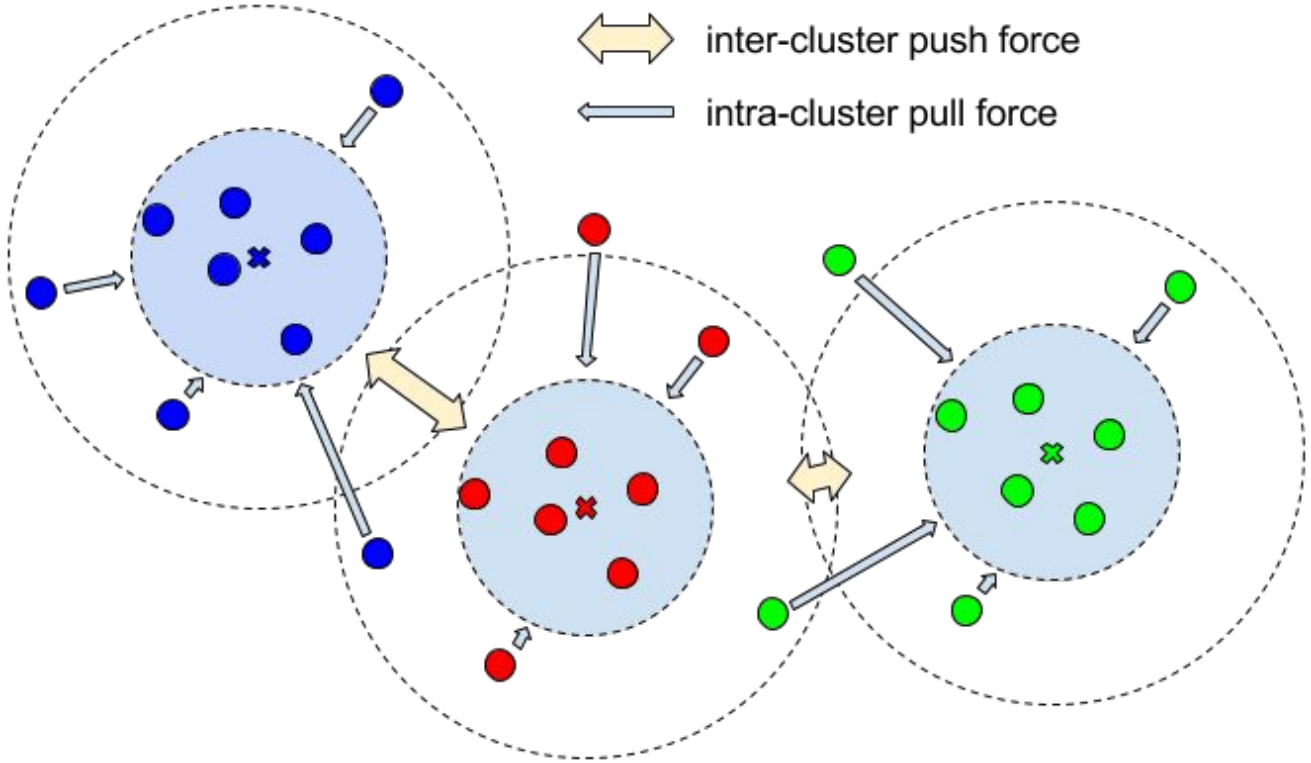
https://arxiv.org/abs/1708.02551

Embedding based segmentation

Loss function:

- **attract pixels that belong to the same instance**
- repel pixels belonging to different instances

$$L_{var} = \frac{1}{C}\sum_{c=1}^{C}\frac{1}{N_c}\sum_{i=1}^{N_c}[\|\mu_c - x_i\| - \delta_v]_+^2$$

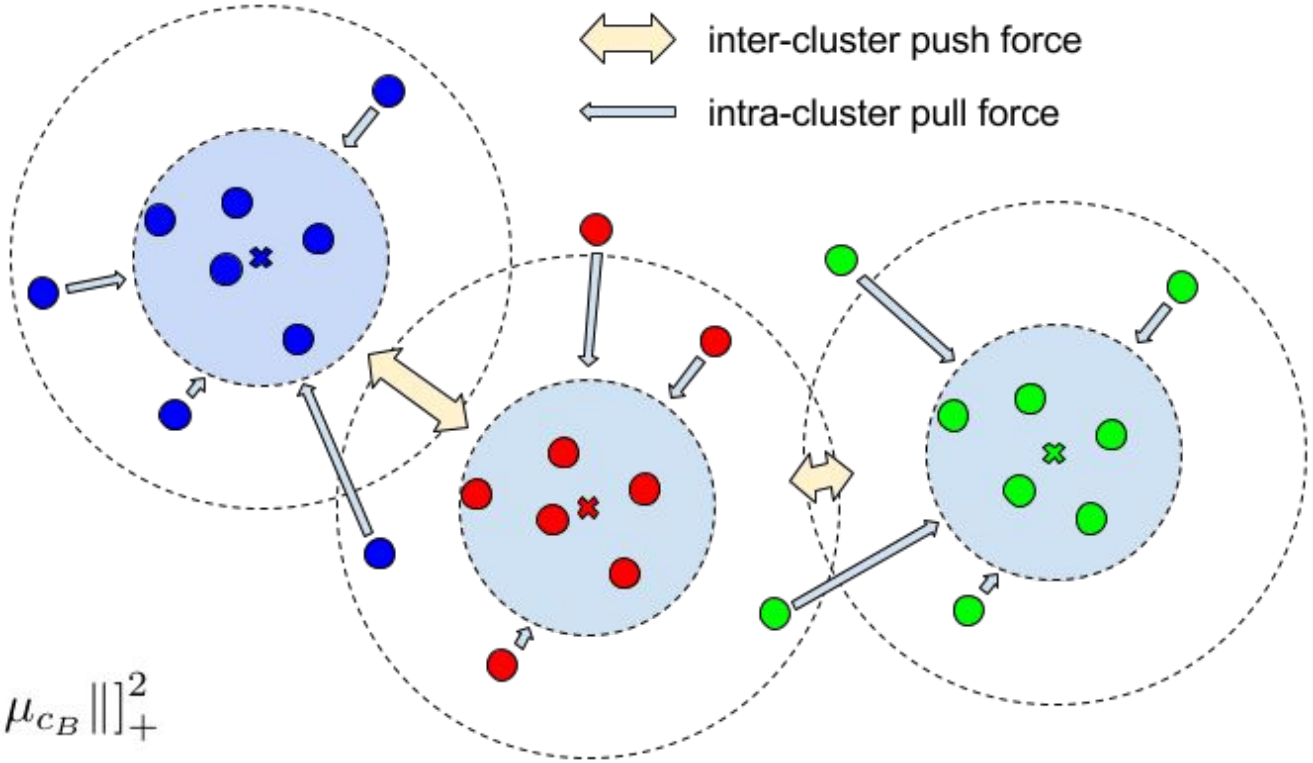


https://arxiv.org/abs/1708.02551

Embedding based segmentation

Loss function:

- attract pixels that belong to the same instance
- **repel pixels belonging to different instances**



$$L_{dist} = \frac{1}{C(C-1)} \sum_{\substack{c_A=1 \\ c_A \neq c_B}}^{C} \sum_{c_B=1}^{C} [2\delta_{\mathrm{d}} - \|\mu_{c_A} - \mu_{c_B}\|]_+^2$$

https://arxiv.org/abs/1708.02551

Embedding based segmentation

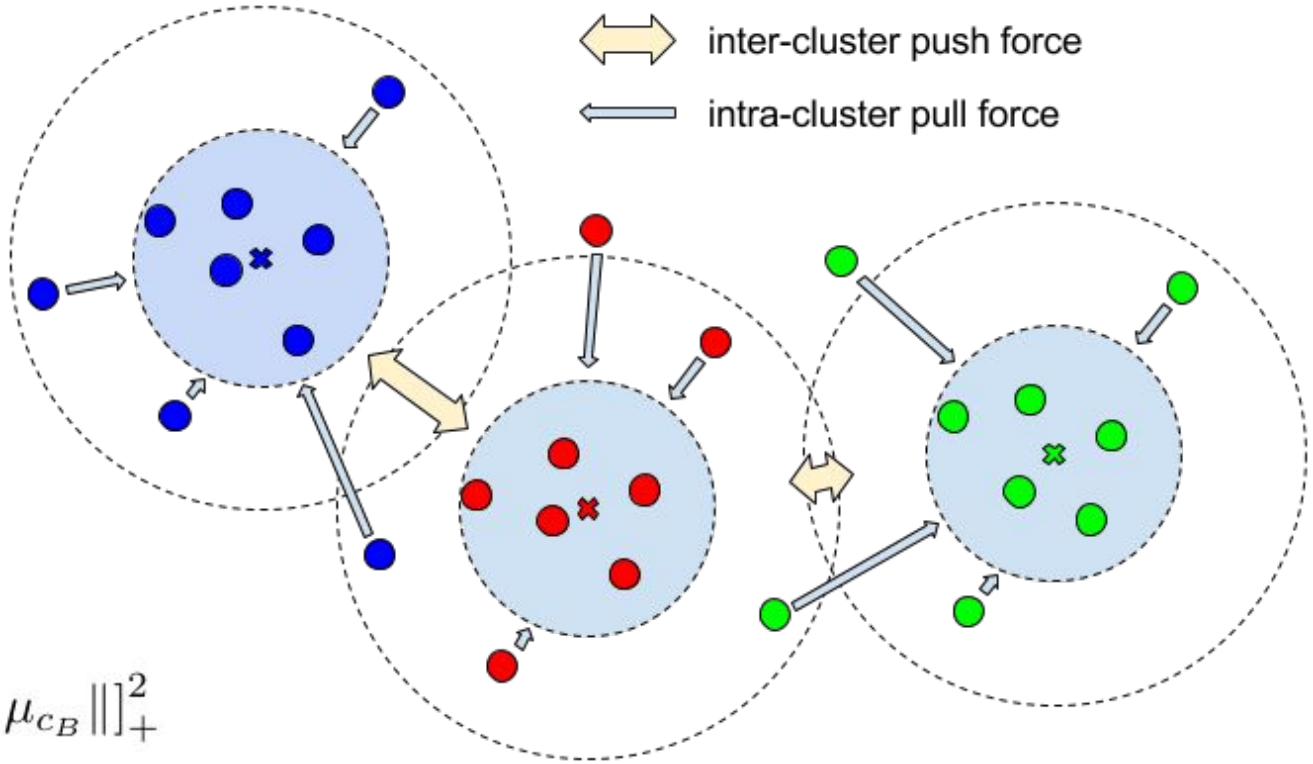
Loss function:

- attract pixels that belong to the same instance
- repel pixels belonging to different instances

$$L = \alpha \cdot L_{var} + \beta \cdot L_{dist}$$

$$L_{var} = \frac{1}{C} \sum_{c=1}^{C} \frac{1}{N_c} \sum_{i=1}^{N_c} [\|\mu_c - x_i\| - \delta_v]_+^2$$

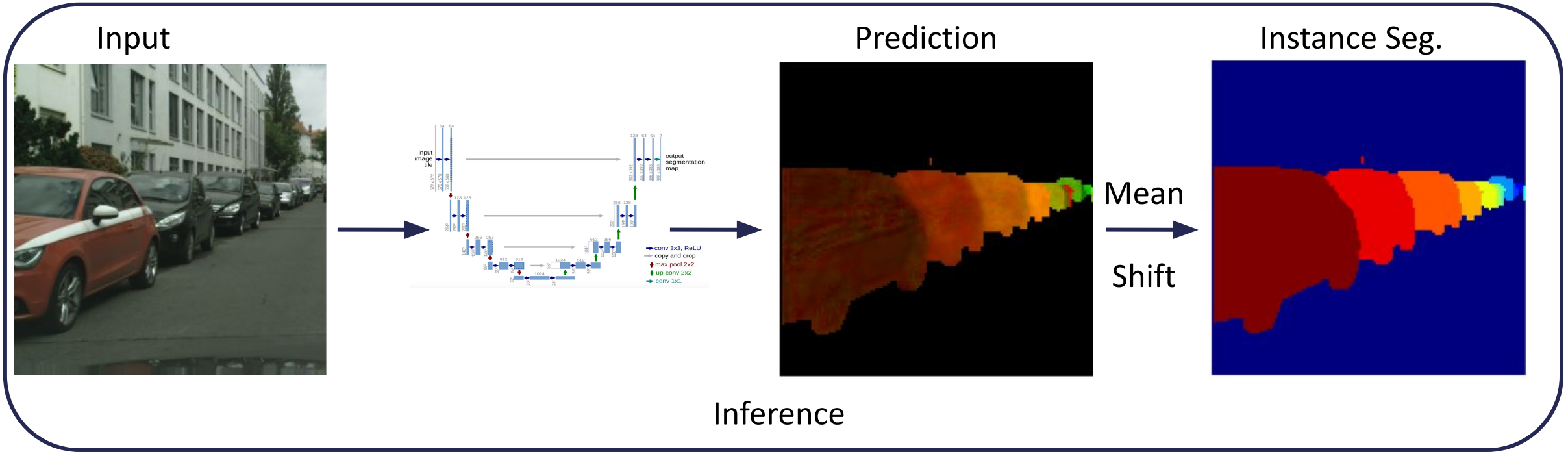
$$L_{dist} = \frac{1}{C(C-1)} \sum_{\substack{c_A=1 \\ c_A \neq c_B}}^{C} \sum_{c_B=1}^{C} [2\delta_d - \|\mu_{c_A} - \mu_{c_B}\|]_+^2$$



https://arxiv.org/abs/1708.02551

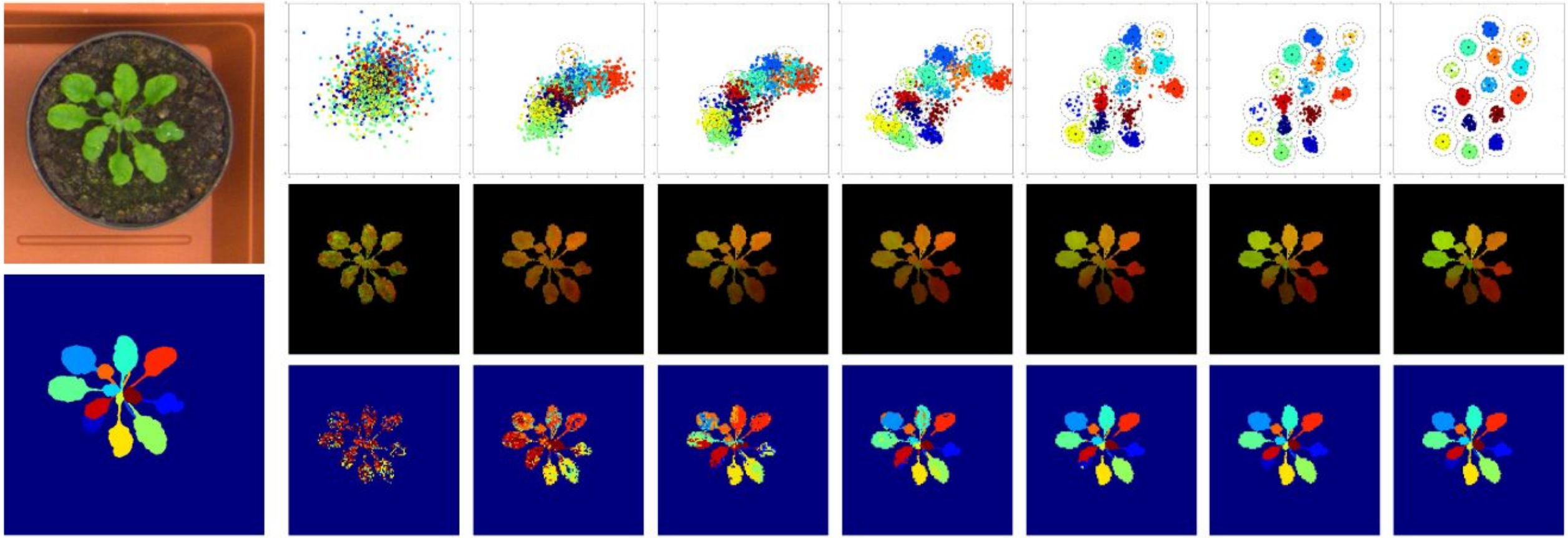
Embedding based segmentation

Find instances using density based clustering, e.g. Mean Shift, HDBSCAN



https://arxiv.org/abs/1708.02551

Embedding based segmentation

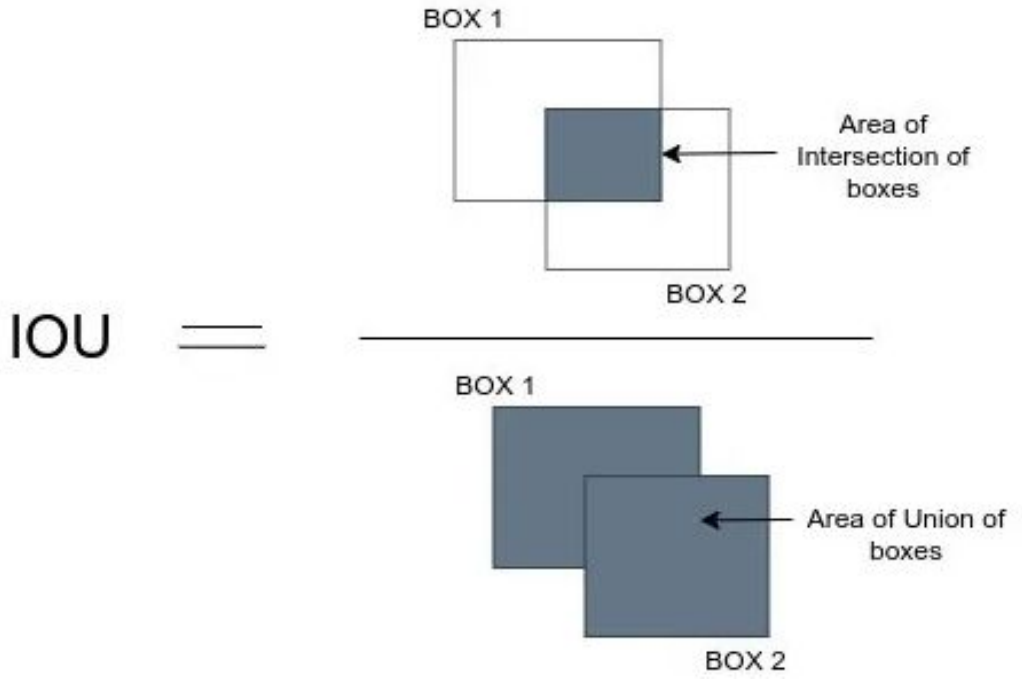


Evaluating object detection and instance segmentation

https://medium.com/analytics-vidhya/iou-intersection-over-union-705a39e7acef

Intersection over Union (IOU):
area of intersection / area of union

Can be formulated for boxes (object detection) or masks (instance segmentation)



Evaluating object detection and instance segmentation

https://medium.com/analytics-vidhya/iou-intersection-over-union-705a39e7acef

Intersection over Union (IOU):
area of intersection / area of union

Can be formulated for boxes (object detection) or masks (instance segmentation)

To evaluate result: define threshold (e.g. 0.5), count all predictions with IOU bigger as correct

Compute Precision, Recall, F1-Score

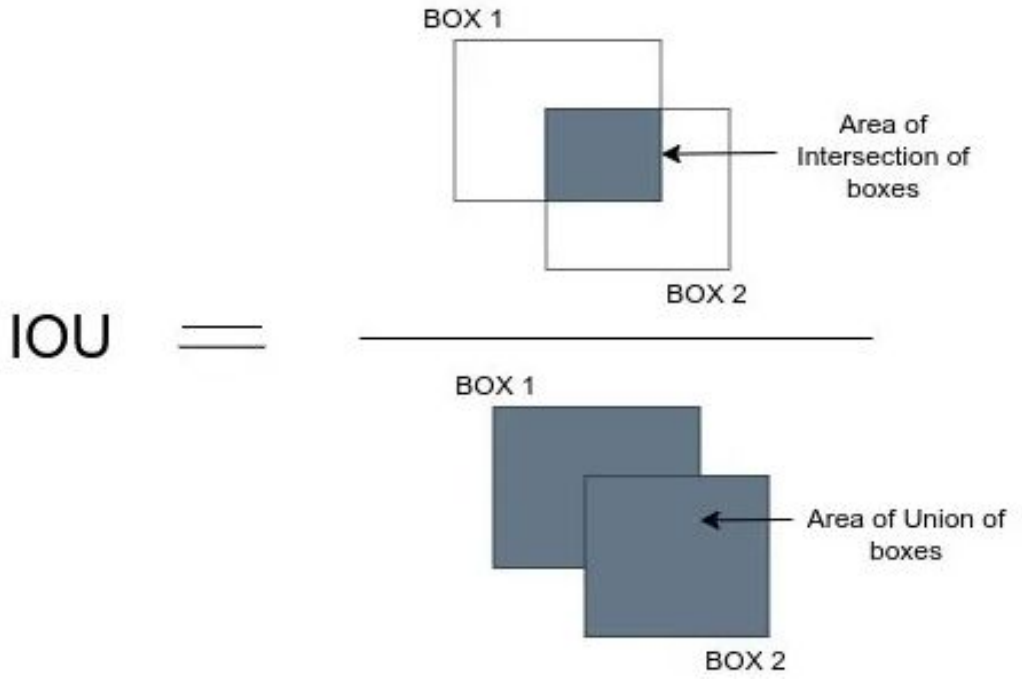
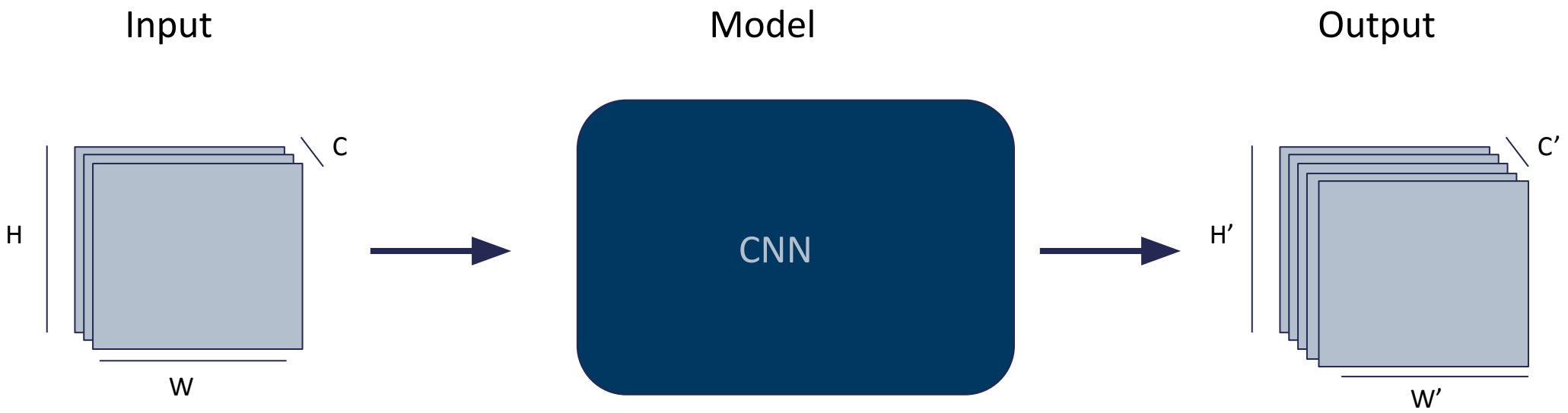


Image-to-image tasks

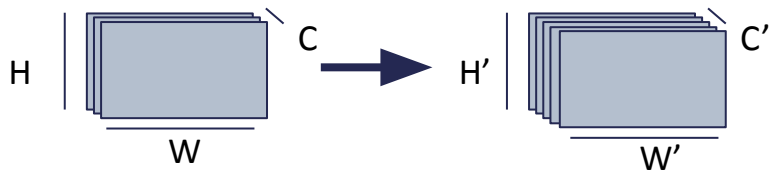
Image-to-image tasks



Input: image with C channels (e.g. RGB)

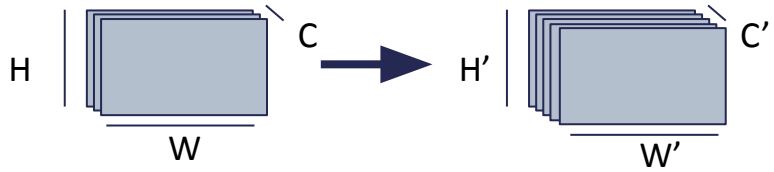
Output: “related” image with number of channels C’ and size H’ x W’

Colorization



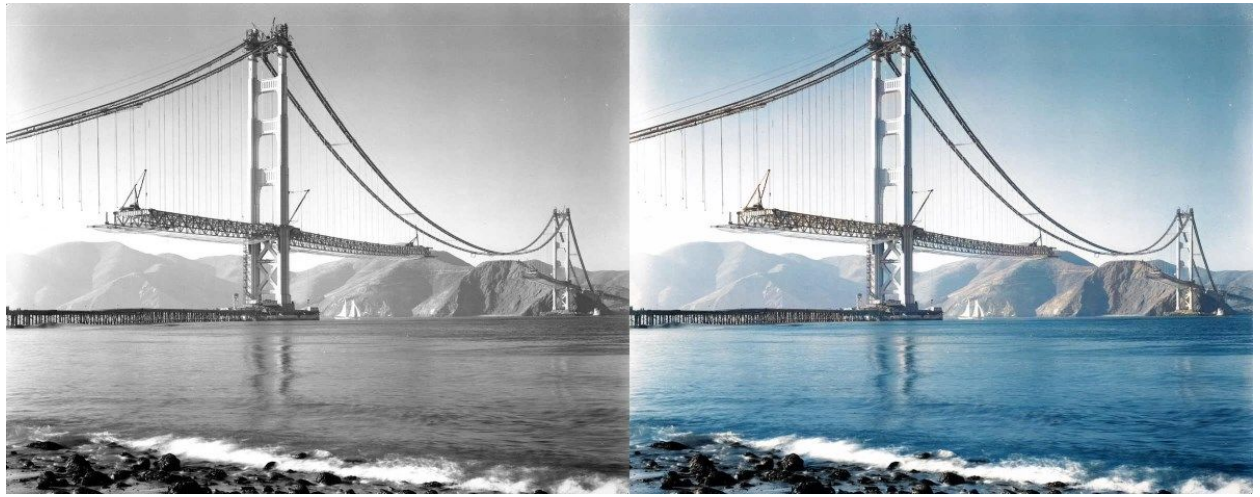
Colorize a grayscale image: **H’=H, W’=W, C=1, C’=3**

Colorization



Colorize a grayscale image: **H'=H, W'=W, C=1, C'=3**

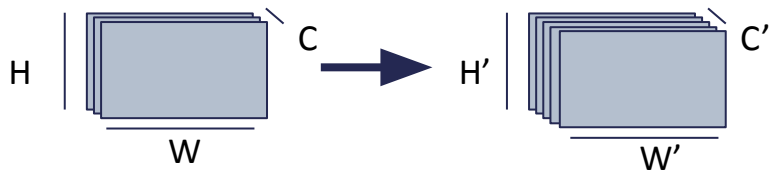
Example method: DeOldify, uses a Generative Adversarial Network (GAN) (Lecture 10)



https://github.com/jantic/DeOldify

Try it yourself:
https://colab.research.google.com/github/jantic/DeOldify/blob/master/ImageColorizerColab.ipynb

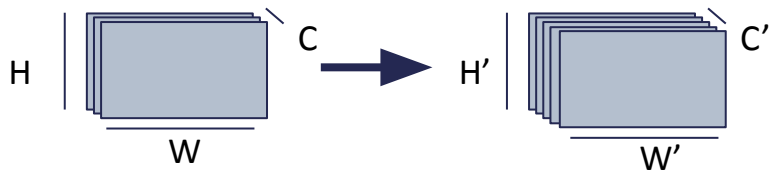
Super-resolution



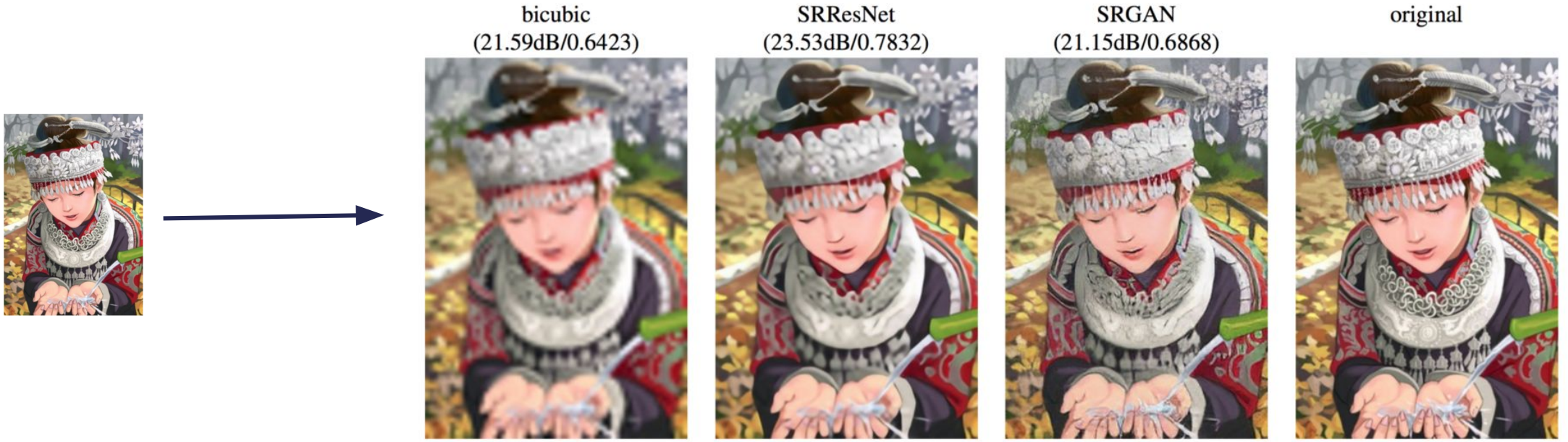
Upsample image by factor **S**: **H’=S*H, W’=S*W, C=C’(=3)**

Can be done with interpolation (e.g. linear or cubic), but leads to blurry images for large **S**

Super-resolution



Upsample image by factor **S**: **H'=S*H, W'=S*W, C=C'(=3)**



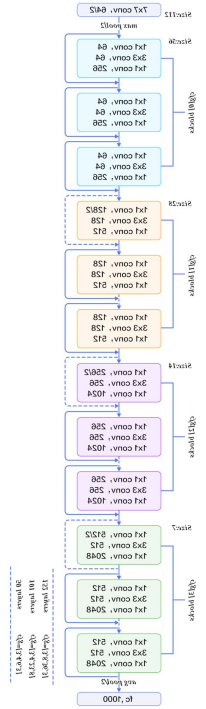
https://medium.com/analytics-vidhya/a-review-on-super-resolution-2c78cd77885a

https://arxiv.org/abs/1609.04802

Super-resolution methods

SRGAN: uses a GAN

SRResNet: ResNet-based architecture trained with MSE / L2 Loss



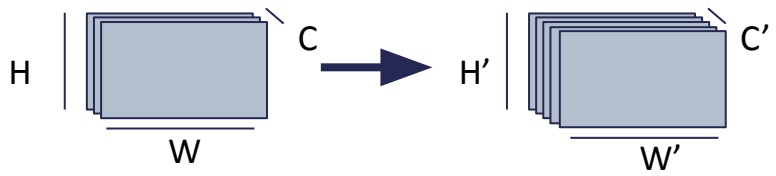
$$\frac{1}{N}\sum_{i}^{N}(P(i) - T(i))^2$$

Input

Prediction *P*

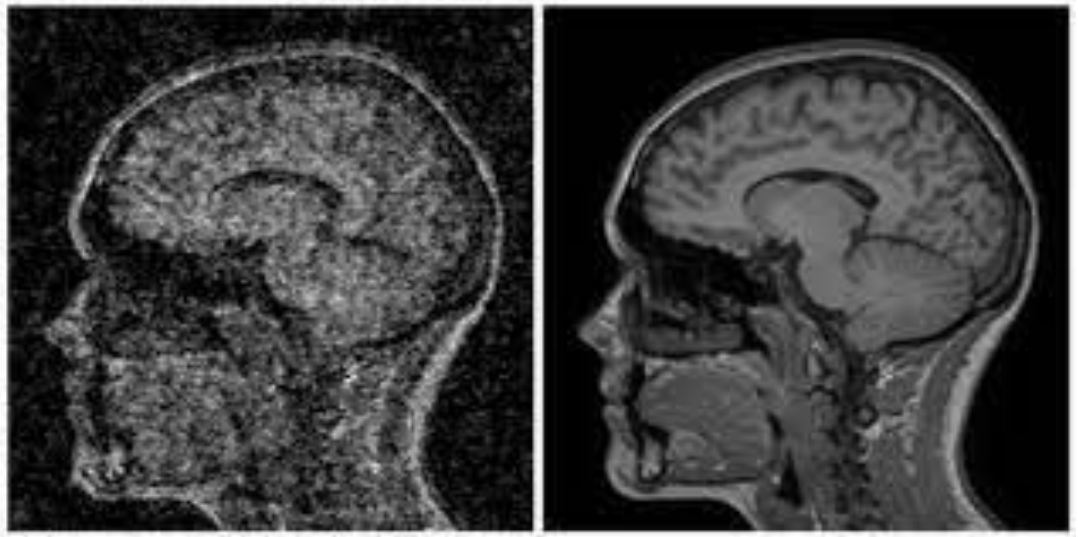
Target *T*

Denoising



Remove noise from the image: **H'=H, W'=W, C=C'**

Very relevant in bio-medical imaging, where
image generation is often noisy (e.g. MRI imaging)



https://www.biorxiv.org/content/10.1101/236463v5

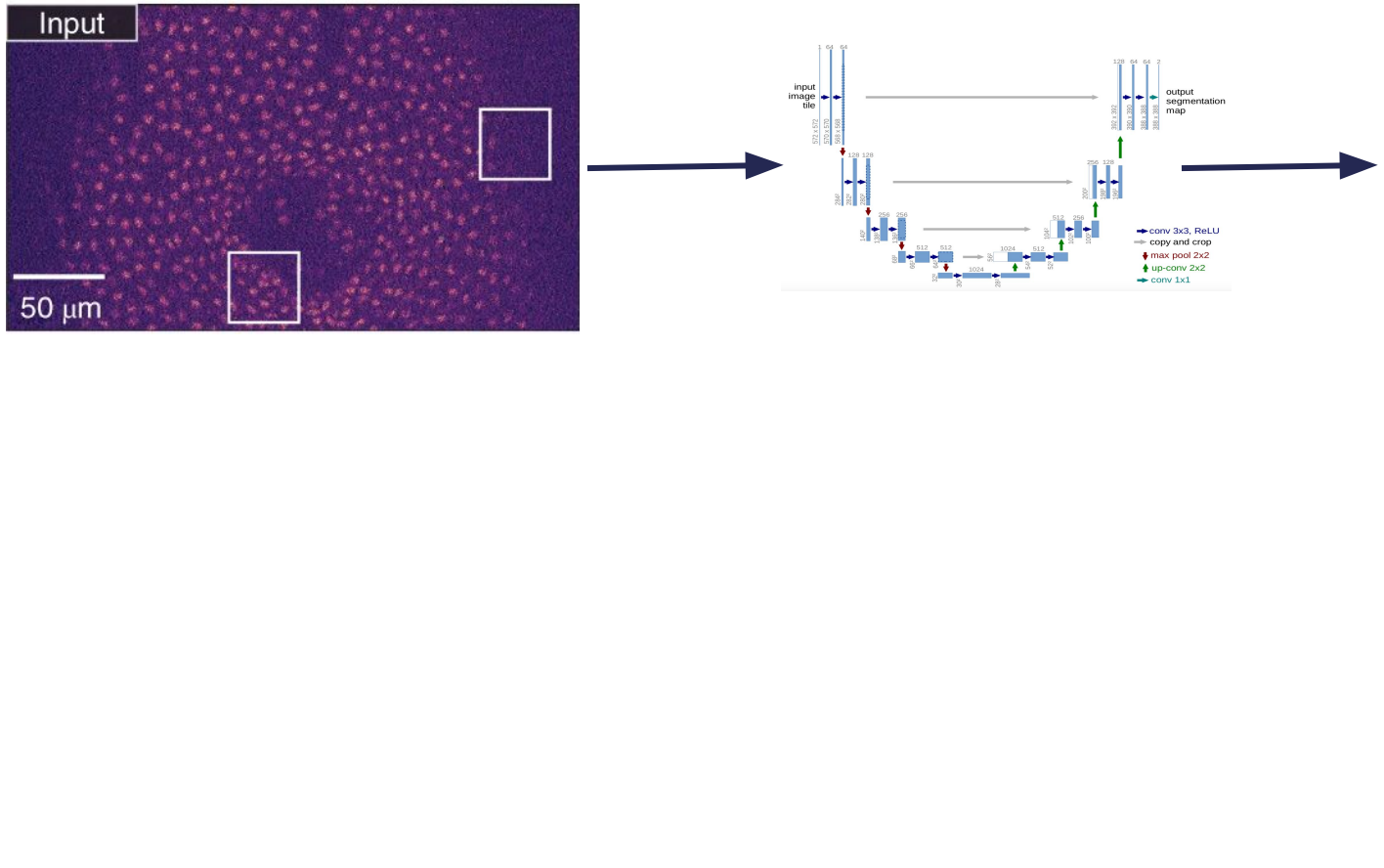
Denoising: CARE

Denoising with Ground-truth data: use U-Net and MSE (L2 Loss)

https://www.biorxiv.org/content/10.1101/236463v5

Denoising: CARE

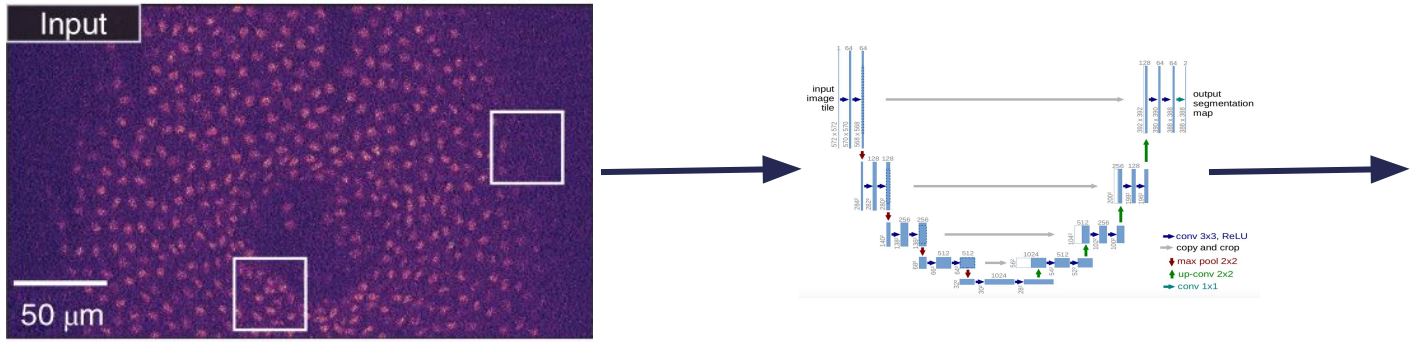
Denoising with Ground-truth data: use U-Net and MSE (L2 Loss)

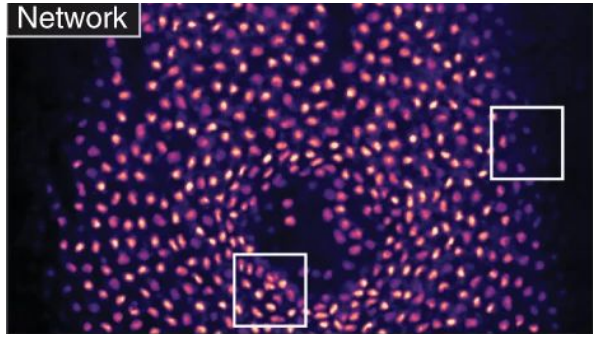


https://www.biorxiv.org/content/10.1101/236463v5

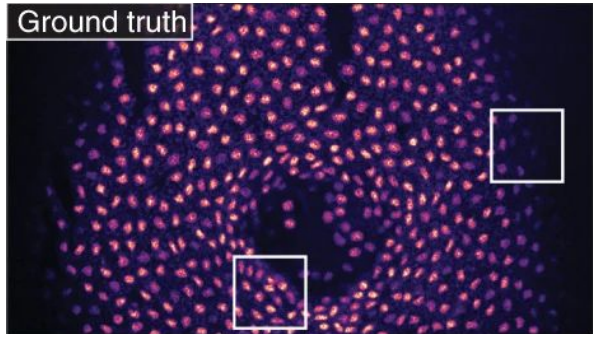
Denoising: CARE

Denoising with Ground-truth data: use U-Net and MSE (L2 Loss)





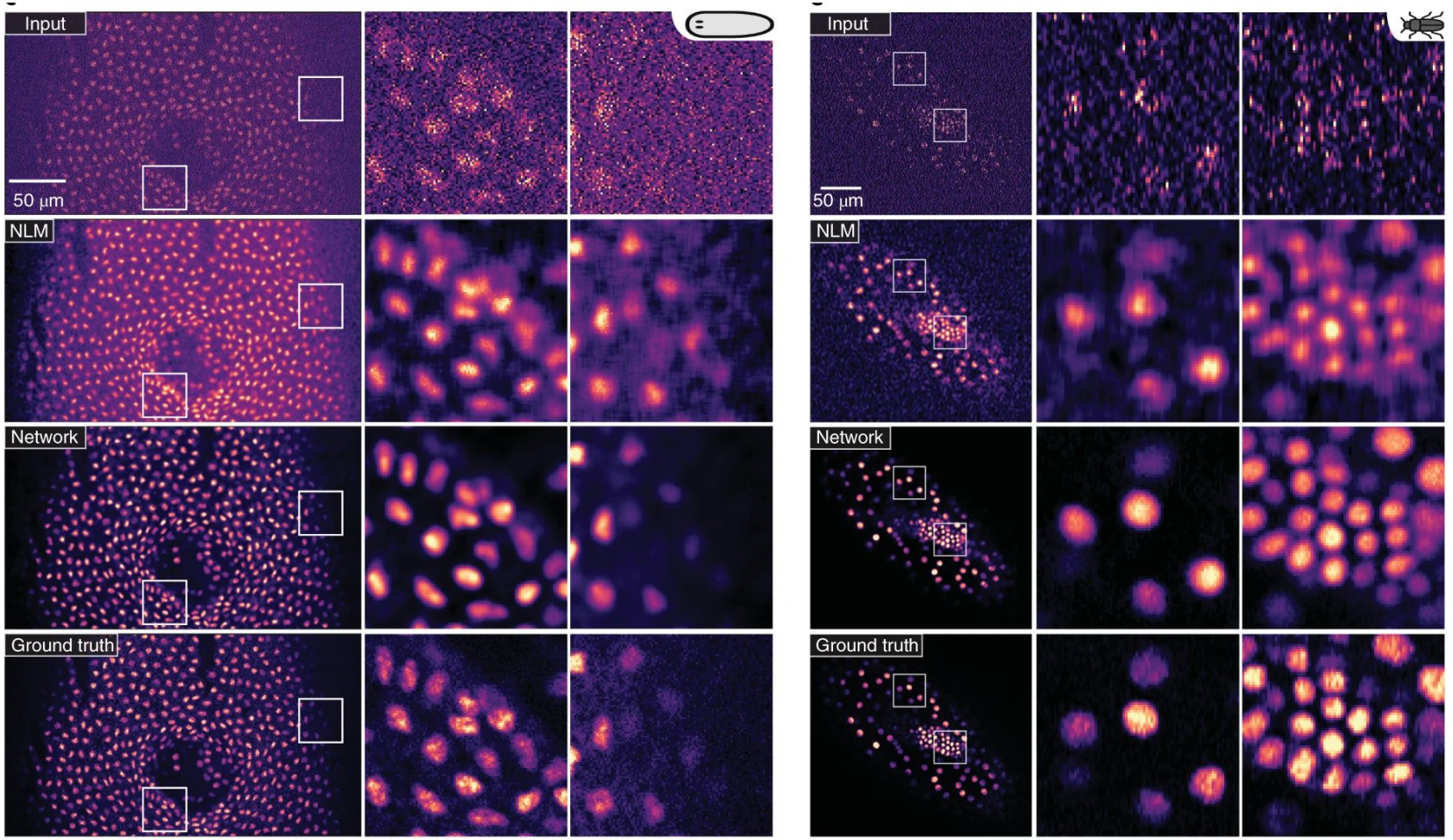
MSE



Microscopy: Ground truth can be generated by the microscope, e.g. with higher laser power (cannot be used for whole experiments, would destroy the sample)

https://www.biorxiv.org/content/10.1101/236463v5

Denoising: CARE



https://arxiv.org/abs/1803.04189

Denoising: Noise to Noise

What if we don’t have Ground truth data?

https://arxiv.org/abs/1803.04189

Denoising: Noise to Noise

What if we don’t have Ground truth data?

Noise to noise: use a different noisy version of image instead!

https://arxiv.org/abs/1803.04189

Denoising: Noise to Noise

What if we don't have Ground truth data?

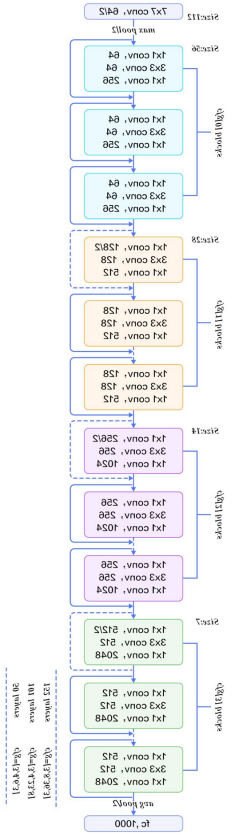
Noise to noise: use a different noisy version of image instead!

How can this work?
MSE loss: estimate remains unchanged if we replace target with random numbers with same expectation value as target

Corrupt training target with zero-mean noise without changing what the network learns!

https://arxiv.org/abs/1803.04189

Denoising: Noise to Noise



MSE

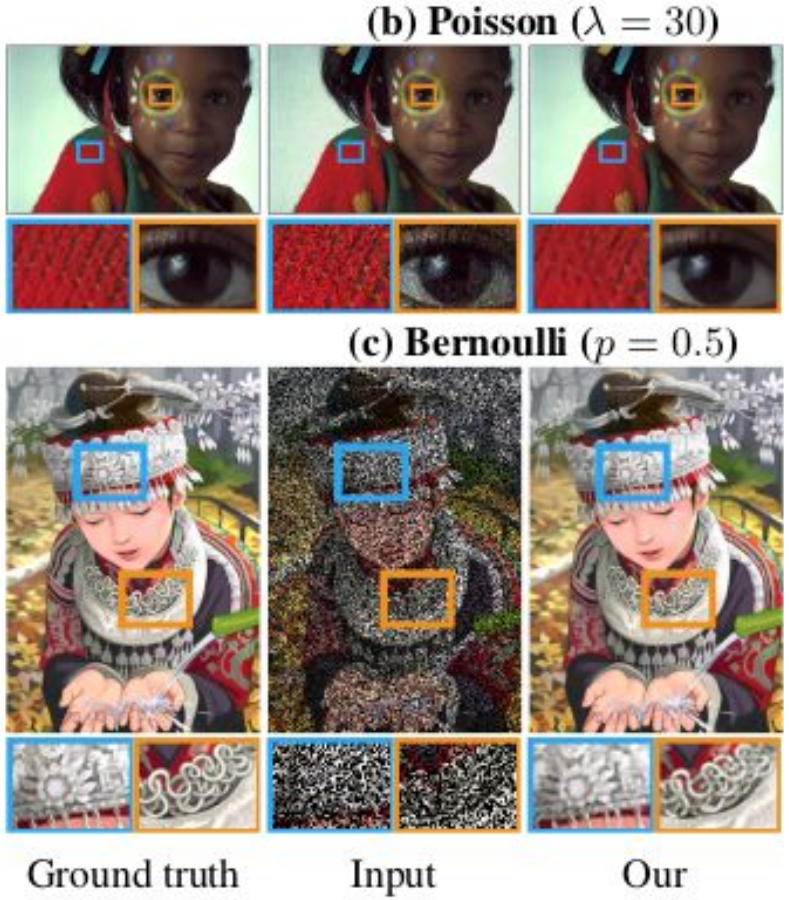
Noisy input

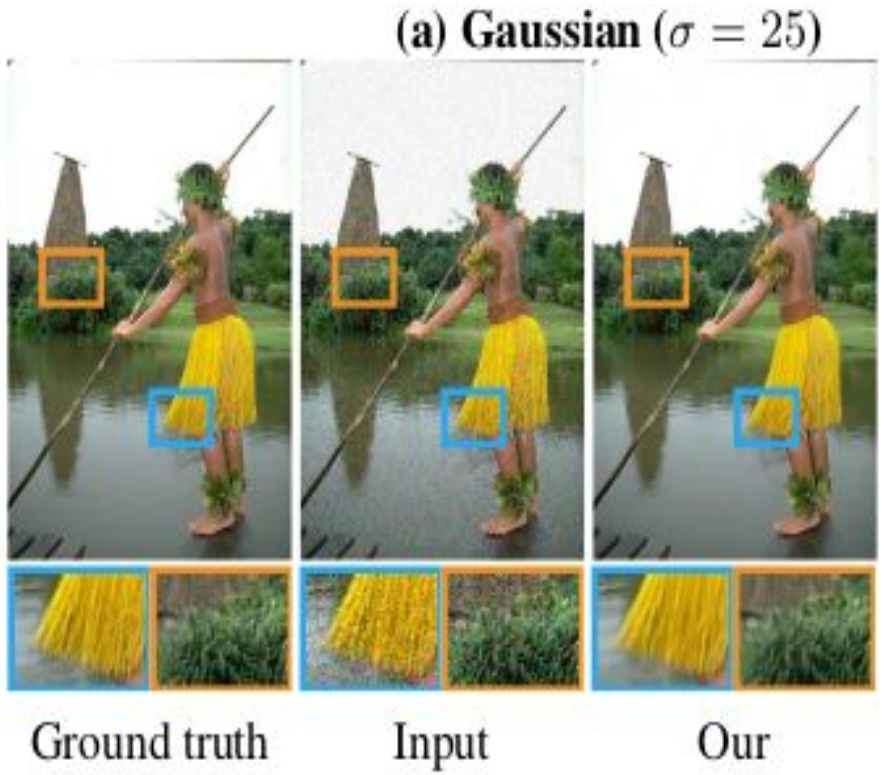
Prediction

Noisy target
(different from input!)

https://arxiv.org/abs/1803.04189

Denoising: Noise to Noise





Questions!